

计算机系统设计PA2实验报告

苏耀磊 2311727

一、实验目的和环境概述

- 掌握 x86 关键指令在 NEMU 中的译码、执行与状态更新方法，建立手册语义与代码实现之间的对应关系。
- 掌握通过 differential testing 与回归测试验证模拟器正确性的基本流程，理解寄存器、标志位与控制流一致性检查方法。
- 在客户程序行为、机器状态与底层执行部件（寄存器、运算逻辑、内存与 I/O 访问）之间建立关联，形成系统级理解。
- 综合运用体系结构与调试知识，分析并解释实现与运行过程中出现的指令语义、数据运算和设备交互相关问题。

本次实验环境使用 vscode 连接 docker 容器，在 debian 中进行实验，实验指导书参考ics2017。

二、实验内容

- 计算机系统设计PA2实验报告
 - 一、实验目的和环境概述
 - 二、实验内容
 - 三、实验过程
 - 1.运行第一个客户程序dummy
 - 1.1 补全 call 指令
 - 1.2 补全 push、pop 指令
 - 1.3 补全 sub 指令
 - 1.4 补全 xor 指令
 - 1.5 补全 ret 指令
 - 问题：立即数背后的故事
 - 2.实现更多的指令
 - 2.1 补全 movzx 和 movsx 指令
 - 2.2 补全 lea 指令
 - 2.3 补全 add 指令
 - 2.4 补全 cmp 指令
 - 2.5 补全 inc 指令
 - 2.6 补全 dec 指令
 - 2.7 补全 neg 指令
 - 2.8 补全 adc 指令
 - 2.9 补全 sbb 指令
 - 2.10 补全 mul 指令
 - 2.11 补全 imul 指令
 - 2.12 补全 div 指令
 - 2.13 补全 idiv 指令
 - 2.14 补全 and 指令

- 2.15 补全 or 指令
 - 2.16 补全 test 指令
 - 2.17 补全 not 指令
 - 2.18 补全 sar、shl、shr 指令
 - 2.19 补全 rol 指令
 - 2.20 补全 setcc 指令
 - 2.21 补全 call_rm 指令
 - 2.22 补全 jmp、jmp_rm 指令
 - 2.23 补全 jcc 指令
 - 2.24 补全 cwtl、cltd 指令
 - 2.25 补全 leave 指令
 - 2.26 补全 nop 指令
 - 2.27 实现 differential testing
 - 2.28 回归测试
 - 问题：堆和栈在哪里？
 - 问题：L语言的三种指令如下，假设除了输入变量之外,其它变量的初值都是0,并且假设程序执行到最后一条指令就结束,你可以仅用这三种指令写一个计算两个正整数相加的程序吗？
 - 问题：NEMU除了作为模拟器之外,还具有简单的调试功能,可以设置断点,查看程序状态.如果让你为NEMU添加如下功能你觉得应该如何实现？
 - 3.运行Hello World
 - 问题：理解volatile关键字
 - 4.实现IOE——时钟
 - 4.1 实现 _uptime() 函数
 - 4.2 看看NEMU跑多快
 - 5.实现IOE(2)——键盘
 - 问题：如何检测多个键同时被按下
 - 6.VGA
 - 6.1 添加内存映射I/O
 - 6.2 实现IOE(3)
 - 6.3 运行打字小游戏
 - 问题：在一些90年代的游戏里,很多渐出渐入效果都是通过调色板实现的,你知道其中的玄机吗？
 - 四、必答题 - 工作方式： - 编译链接过程：
 - 五、实验过程中的问题与解决方法
-

三、实验过程

1.运行第一个客户程序dummy

首先进入目录 nexus-am/tests/cputest，键入

```
make ARCH=x86-nemu ALL=dummy run
```

结果如下：

```

SuYaolei@988d7f56373a:~/code/ics2017/nexus-am/tests/cputest$ make ARCH=x86-nemu ALL=dummy run
[src/monitor/monitor.c,65,load_img] The image is /home/SuYaolei/code/ics2017/nexus-am/tests/cp
Welcome to NEMU!
[src/monitor/monitor.c,30,welcome] Build time: 19:05:03, Apr 20 2026
For help, type "help"
(nemu) c
invalid opcode(eip = 0x0010000a): e8 01 00 00 00 90 55 89 ...

There are two cases which will trigger this unexpected exception:
1. The instruction at eip = 0x0010000a is not implemented.
2. Something is implemented incorrectly.
Find this eip(0x0010000a) in the disassembling result to distinguish which case it is.

If it is the first case, see

for more details.

If it is the second case, remember:
* The machine is always right!
* Every line of untested code is always wrong!

(nemu) █

```

观察报错，发现是0x10000a处指令出现问题，操作码是e8，从反汇编结果 (nexus-am/tests/cputest/build/dummy-x86-nemu.txt) 中可以验证是 call 指令没有实现导致的，当然后面实现了 call 以后还会在其他位置报错，因此在第一阶段，想要运行 dummy ，就必须补全它所涉及到的指令。

1.1 补全 call 指令

因为上面图的报错显示是指令码是e8，从i386手册中的“page 414 of 421”的表格的第E行第8列可知是CALL，当然也可以通过反汇编的地址得到是哪条指令有问题，如下：

E	LOOPNE	LOOPE	LOOP	JCXZ	IN		OUT		CALL
	Jb	Jb	Jb	Jb	AL, Ib	eAX, Ib	Ib, AL	Ib, eAX	Av

而根据手册上一页的内容我们也能查到 A 和 v 的含义如下：

A Direct address; the instruction has no modR/M byte; the address of the operand is encoded in the instruction; no base register, index register, or scaling factor can be applied; e.g., far JMP (EA).

v Word or double word, depending on operand size attribute.

这里 A 代表直接寻址，v 代表单字或双字，继续搜索e8，在手册的275页找到了call指令介绍内容以及它的伪代码：

E8	cw	CALL rel16	7+m	Call near, displacement relative to next instruction
FF	/2	CALL r/m16	7+m/10+m	Call near, register indirect/memory indirect
9A	cd	CALL ptr16:16	17+m, pm=34+m	Call intersegment, to full pointer given
9A	cd	CALL ptr16:16	pm=52+m	Call gate, same privilege
9A	cd	CALL ptr16:16	pm=86+m	Call gate, more privilege, no parameters
9A	cd	CALL ptr16:16	pm=94+4x+m	Call gate, more privilege, x parameters
9A	cd	CALL ptr16:16	ts	Call to task
FF	/3	CALL m16:16	22+m, pm=38+m	Call intersegment, address at r/m dword
FF	/3	CALL m16:16	pm=56+m	Call gate, same privilege
FF	/3	CALL m16:16	pm=90+m	Call gate, more privilege, no parameters
FF	/3	CALL m16:16	pm=98+4x+m	Call gate, more privilege, x parameters
FF	/3	CALL m16:16	5 + ts	Call to task
E8	cd	CALL rel32	7+m	Call near, displacement relative to next instruction
FF	/2	CALL r/m32	7+m/10+m	Call near, indirect
9A	cp	CALL ptr16:32	17+m, pm=34+m	Call intersegment, to pointer given
9A	cp	CALL ptr16:32	pm=52+m	Call gate, same privilege
9A	cp	CALL ptr16:32	pm=86+m	Call gate, more privilege, no parameters
9A	cp	CALL ptr32:32	pm=94+4x+m	Call gate, more privilege, x parameters
9A	cp	CALL ptr16:32	ts	Call to task
FF	/3	CALL m16:32	22+m, pm=38+m	Call intersegment, address at r/m dword
FF	/3	CALL m16:32	pm=56+m	Call gate, same privilege
FF	/3	CALL m16:32	pm=90+m	Call gate, more privilege, no parameters
FF	/3	CALL m16:32	pm=98+4x+m	Call gate, more privilege, x parameters
FF	/3	CALL m16:32	5 + ts	Call to task

```

IF rel16 or rel32 type of call
THEN (* near relative call *)
    IF OperandSize = 16
    THEN
        Push(IP);
        EIP ← (EIP + rel16) AND 0000FFFFH;
    ELSE (* OperandSize = 32 *)
        Push(EIP);
        EIP ← EIP + rel32;
    FI;
FI;

```

接下来，就可以在 nemu/src/cpu/exec/exec.c 的 opcode_table 里补充对应的代码了，当然除此之外，对于需要用到的宏 IDEX 和 IDEXW 做一个简单的解释：

```

10  #define IDEXW(id, ex, w) {concat(decode_, id), concat(exec_, ex), w}
11  #define IDEX(id, ex)    IDEXW(id, ex, 0)

```

IDEXW 的三个参数是译码所需要的函数、执行的函数以及位宽，而 IDEX 就是 IDEXW 的位宽为 0 的情况。

我们知道 call 是一种带有返回地址的跳转，所以可以使用 J 作为译码函数，再执行 call 的操作，因此我们补充 opcode_table 的 0xe8 的代码如下：

```

134  /* 0xe8 */ IDEX(J, call), IDEX(J, jmp), IDEX(I, jmp_rm), IDEXW(J, jmp, 1),

```

而 call 的操作在 nemu/src/cpu/exec/control.c 里，补充 make_EHelper(call) 如下：

```

26  make_EHelper(call) {
27      // the target address is calculated at the decode stage
28      rtl_li(&t0, *eip);
29      rtl_push(&t0);
30      decoding.is_jump = 1;
31
32      print_asm("call %x", decoding.jump_eip);
33  }

```

这里完成 call 相比 jmp 多出来的保存返回地址的部分，rtl_li(&t0, *eip) 是把当前的 eip（实际上是下一条指令地址）读出来放到临时寄存器 t0 中。而因为 *eip 已经被更新为当前指令执行完之后的地址，所以它正好就是 call 返回时应该跳回的位置。接下来 rtl_push(&t0) 把这个地址压入栈中，然后 decoding.is_jump = 1 控制后续不会再按照顺序执行下一条指令，而是跳转到目标地址。

之后需要在 nemu/src/cpu/exec/all-instr.h 文件中添加对应的声明，如下：

```

34  make_EHelper(call);
35  make_EHelper(call_rm);
36  make_EHelper(ret);
37  make_EHelper(jmp);
38  make_EHelper(jmp_rm);
39  make_EHelper(jcc);
40  make_EHelper(cwtl);
41  make_EHelper(cltd);

```

在 PA2 后面的部分还会添加大量指令，都需要在这里添加声明，因此后面不再赘述。

到这里 call 指令的添加就算完成了，我们再次运行程序后就不会在这里中断了，会继续执行到下一条还没有补充的指令，我们只需要按照上面所说的流程，补完所有需要用到的指令就可以成功运行 dummy 了。

1.2 补全 push、pop 指令

按照指导书来说，可以很容易地通过 rtl_push 和 rtl_pop 来实现，在 nemu/include/cpu/rtl.h 里，实现如下：

```

148 static inline void rtl_push(const rtlreg_t* src1) {
149     // esp <- esp - 4
150     // M[esp] <- src1
151     rtl_subi(&cpu.esp, &cpu.esp, 4);
152     rtl_sm(&cpu.esp, 4, src1);
153 }
154
155 static inline void rtl_pop(rtlreg_t* dest) {
156     // dest <- M[esp]
157     // esp <- esp + 4
158     rtl_lm(dest, &cpu.esp, 4);
159     rtl_addi(&cpu.esp, &cpu.esp, 4);
160 }

```

除此之外，补充 make_EHelper(push) 和 make_EHelper(pop) 如下：

```

8 make_EHelper(push) {
9     rtl_push(&id_dest->val);
10
11     print_asm_template1(push);
12 }
13
14 make_EHelper(pop) {
15     rtl_pop(&t0);
16     operand_write(id_dest, &t0);
17
18     print_asm_template1(pop);
19 }

```

还有 opcode_table 里补充如下：

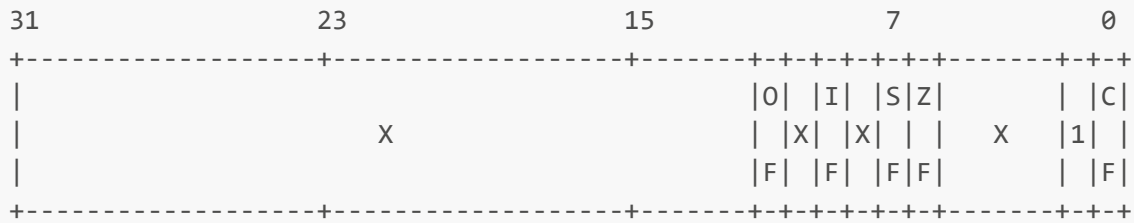
```

95 /* 0x50 */ IDEX(r,push), IDEX(r,push), IDEX(r,push), IDEX(r,push),
96 /* 0x54 */ IDEX(r,push), IDEX(r,push), IDEX(r,push), IDEX(r,push),
97 /* 0x58 */ IDEX(r,pop), IDEX(r,pop), IDEX(r,pop), IDEX(r,pop),
98 /* 0x5c */ IDEX(r,pop), IDEX(r,pop), IDEX(r,pop), IDEX(r,pop),

```

1.3 补全 sub 指令

接下来是 sub 指令，在补全sub之前要先实现 EFLAGS 寄存器，根据指导书，它的结构如下：



在我们 pa1 实现的的寄存器结构里，也就是 nemu/include/cpu/reg.h 补充如下：

```

35     union {
36         rtlreg_t eflags;
37         struct {
38             uint32_t CF : 1;
39             uint32_t     : 1;
40             uint32_t PF : 1;
41             uint32_t     : 1;
42             uint32_t AF : 1;
43             uint32_t     : 1;
44             uint32_t ZF : 1;
45             uint32_t SF : 1;
46             uint32_t TF : 1;
47             uint32_t IF : 1;
48             uint32_t DF : 1;
49             uint32_t OF : 1;
50             uint32_t     : 20;
51         };
52     };

```

除此之外，根据指导书所说，关于 EFLAGS 的初值，我们遵循i386手册中提到的约定，在 restart() 函数中对 EFLAGS寄存器进行初始化为 0x2 ，如下：

```

82  static inline void restart() {
83      /* Set the initial instruction pointer. */
84      cpu.eip = ENTRY_START;
85      /* Init the rest of CPU states. */
86      cpu.eflags = 0x2;
87      #ifdef DIFF_TEST
88      |   init_gemu_reg();
89      #endif
90  }

```

之后完善译码的辅助函数 `make_DopHelper(SI)`，在 `nemu/src/cpu/decode/decode.c` 里，根据注释实现如下：

```

30  static inline make_DopHelper(SI) {
31      assert(op->width == 1 || op->width == 4);
32
33      op->type = OP_TYPE_IMM;
34
35      /* TODO: Use instr_fetch() to read `op->width' bytes of memory
36       * pointed by `eip'. Interpret the result as a signed immediate,
37       * and assign it to op->simm.
38       *
39       op->simm = ???
40       */
41
42      if (op->width == 1) {
43          op->simm = (int32_t)(int8_t)instr_fetch(eip, 1);
44      }
45      else {
46          op->simm = (int32_t)instr_fetch(eip, 4);
47      }
48
49      rtl_li(&op->val, op->simm);
50
51      #ifdef DEBUG
52      |   snprintf(op->str, OP_STR_SIZE, "$0x%x", op->simm);
53      #endif
54  }

```

接下来我们需要补充一些rtl指令来方便后续的其他指令的使用，像上面的 `push` 和 `pop` 一样。

在NEMU中，RTL指令有两种(在 `nemu/include/cpu/rtl.h` 中定义)。一种是RTL基本指令，它们的特点是在即时编译技术里面可以只使用一条机器指令来实现相应的功能，同时也不需要临时寄存器，可以看做是最基本的x86指令中的最基本的操作。RTL基本指令包括：

- 立即数读入rtl_li
- 算术运算和逻辑运算, 包括寄存器-寄存器类型rtl_(add|sub|and|or|xor|shl|shr|sar|slt|sltu) 和立即数-寄存器类型rtl_(add|sub|and|or|xor|shl|shr|sar|slt|sltu)i 内存的访存rtl_lm和rtl_sm
- 通用寄存器的访问rtl_lr_(b|w|l)和rtl_sr_(b|w|l)

第二种RTL指令是RTL伪指令, 它们是通过RTL基本指令或者已经实现的RTL伪指令来实现的, 包括:

- 带宽度的通用寄存器访问rtl_lr和rtl_sr
- EFLAGS标志位的读写rtl_set_(CF|OF|ZF|SF|IF)和rtl_get_(CF|OF|ZF|SF|IF)
- 其它常用功能, 如数据移动rtl_mv, 符号扩展rtl_sext等

实现的如下:

```
132  static inline void rtl_not(rtlreg_t* dest) {
133      // dest <- ~dest
134      *dest = ~(*dest);
135  }
136
137  static inline void rtl_sext(rtlreg_t* dest, const rtlreg_t* src1, int width) {
138      // dest <- signext(src1[(width * 8 - 1) .. 0])
139      assert(width == 1 || width == 2 || width == 4);
140      switch (width) {
141          case 1: *dest = (int32_t)(int8_t)(*src1 & 0xff); break;
142          case 2: *dest = (int32_t)(int16_t)(*src1 & 0xffff); break;
143          case 4: *dest = *src1; break;
144          default: assert(0);
145      }
146  }
```

```
162  static inline void rtl_eq0(rtlreg_t* dest, const rtlreg_t* src1) {
163      // dest <- (src1 == 0 ? 1 : 0)
164      *dest = (*src1 == 0);
165  }
166
167  static inline void rtl_eqi(rtlreg_t* dest, const rtlreg_t* src1, int imm) {
168      // dest <- (src1 == imm ? 1 : 0)
169      *dest = (*src1 == (rtlreg_t)imm);
170  }
171
172  static inline void rtl_neq0(rtlreg_t* dest, const rtlreg_t* src1) {
173      // dest <- (src1 != 0 ? 1 : 0)
174      *dest = (*src1 != 0);
175  }
176
177  static inline void rtl_msb(rtlreg_t* dest, const rtlreg_t* src1, int width) {
178      // dest <- src1[width * 8 - 1]
179      assert(width == 1 || width == 2 || width == 4);
180      *dest = (*src1 >> (width * 8 - 1)) & 0x1;
181  }
```

```

183 static inline void rtl_update_ZF(const rtlreg_t* result, int width) {
184     // eflags.ZF <- is_zero(result[width * 8 - 1 .. 0])
185     assert(width == 1 || width == 2 || width == 4);
186     rtlreg_t mask = (width == 4 ? 0xffffffffu : ((1u << (width * 8)) - 1));
187     t0 = ((*result & mask) == 0);
188     rtl_set_ZF(&t0);
189 }
190
191 static inline void rtl_update_SF(const rtlreg_t* result, int width) {
192     // eflags.SF <- is_sign(result[width * 8 - 1 .. 0])
193     rtl_msb(&t0, result, width);
194     rtl_set_SF(&t0);
195 }
196
197 static inline void rtl_update_ZFSF(const rtlreg_t* result, int width) {
198     rtl_update_ZF(result, width);
199     rtl_update_SF(result, width);
200 }

```

接下来就可以实现 sub 指令了，参考 i386 手册 page404 得到：

SUB — Integer Subtraction

Opcode	Instruction	Clocks	Description
2C ib	SUB AL,imm8	2	Subtract immediate byte from AL
2D iw	SUB AX,imm16	2	Subtract immediate word from AX
2D id	SUB EAX,imm32	2	Subtract immediate dword from EAX
80 /5 ib	SUB r/m8,imm8	2/7	Subtract immediate byte from r/m byte
81 /5 iw	SUB r/m16,imm16	2/7	Subtract immediate word from r/m word
81 /5 id	SUB r/m32,imm32	2/7	Subtract immediate dword from r/m dword
83 /5 ib	SUB r/m16,imm8	2/7	Subtract sign-extended immediate byte from r/m word
83 /5 ib	SUB r/m32,imm8	2/7	Subtract sign-extended immediate byte from r/m dword
28 /r	SUB r/m8,r8	2/6	Subtract byte register from r/m byte
29 /r	SUB r/m16,r16	2/6	Subtract word register from r/m word
29 /r	SUB r/m32,r32	2/6	Subtract dword register from r/m dword
2A /r	SUB r8,r/m8	2/7	Subtract byte register from r/m byte
2B /r	SUB r16,r/m16	2/7	Subtract word register from r/m word
2B /r	SUB r32,r/m32	2/7	Subtract dword register from r/m dword

从整体上看，这些 SUB 指令可以分为三类。

第一类是固定寄存器与立即数的形式，例如对 AL 或 EAX 做减法，这类指令编码不需要额外的 ModR/M 字节。

第二类是寄存器或内存（r/m）与立即数的形式，这类指令需要通过 ModR/M 字节确定操作数位置，其中有一种特殊情况（opcode 为 83）需要对 8 位立即数进行符号扩展。

第三类是寄存器和寄存器/内存之间的减法，这类指令的关键在于区分谁是目标操作数。

此外，表中的 /r 和 /5 表示指令编码方式：/r 说明需要用 ModR/M 字节来表示两个操作数，而 /5 表示该指令属于扩展指令。

还有 i386 手册 page418 的表格来告知各个 eflag 位是如何变化的，后面就不再赘述，如下：

Key to Codes

T = instruction tests flag
M = instruction modifies flag
(either sets or resets depending on operands)
0 = instruction resets flag
1 = instruction sets flag
— = instruction's effect on flag is undefined
R = instruction restores prior value of flag
blank = instruction does not affect flag

Instruction	OF	SF	ZF	AF	PF	CF	TF	IF	DF	NT	RF
STI								1			
STOS									T		
STR											
SUB	M	M	M	M	M	M					
TEST	0	M	M	—	M	0					
VERR/VERRW			M								
WAIT											

因此根据这些，实现的 make_EHelper(sub) 如下：

```

22  make_EHelper(sub) {
23      rtl_sub(&t2, &id_dest->val, &id_src->val);
24      operand_write(id_dest, &t2);
25
26      rtl_update_ZFSF(&t2, id_dest->width);
27
28      rtl_sltu(&t0, &id_dest->val, &id_src->val);
29      rtl_set_CF(&t0);
30
31      rtl_xor(&t0, &id_dest->val, &id_src->val);
32      rtl_xor(&t1, &id_dest->val, &t2);
33      rtl_and(&t0, &t0, &t1);
34      rtl_msb(&t0, &t0, id_dest->width);
35      rtl_set_OF(&t0);
36
37      print_asm_template2(sub);
38  }

```

补充的 opcode_table 如下：

```

85  /* 0x28 */ IDEXW(G2E, sub, 1), IDEX(G2E, sub), IDEXW(E2G, sub, 1), IDEX(E2G, sub),
86  /* 0x2c */ IDEXW(I2a, sub, 1), IDEX(I2a, sub), EMPTY, EMPTY,

107 /* 0x80 */ IDEXW(I2E, gp1, 1), IDEX(I2E, gp1), EMPTY, IDEX(SI2E, gp1),

```

这里其实就是利用宏IDEX和IDEXW来对应上面的表格，例如，G2E、E2G 表示不同的数据流方向，也就是寄存器到 r/m，或 r/m 到寄存器，对应寄存器与内存之间的减法操作；I2a 表示立即数到累加器，对应最简单的立即数减法；而 I2E、SI2E 则表示立即数到 r/m 的情况。

对于 0x80 / 0x81 / 0x83 这类 opcode，由于它们需要扩展，因此查看 make_group 函数，补充如下：

```

42  /* 0x80, 0x81, 0x83 */
43  make_group(gp1,
44      EX(add), EX(or), EX adc), EX(sbb),
45      EX(and), EX(sub), EX(xor), EX(cmp))

```

根据表格内容将sub填到5号位置上。

1.4 补全 xor 指令

接下来是 xor 指令，查阅 i386 手册得到：

XOR — Logical Exclusive OR

Opcode	Instruction	Clocks	Description
34 ib	XOR AL,imm8	2	Exclusive-OR immediate byte to AL
35 iw	XOR AX,imm16	2	Exclusive-OR immediate word to AX
35 id	XOR EAX,imm32	2	Exclusive-OR immediate dword to EAX
80 /6 ib	XOR r/m8,imm8	2/7	Exclusive-OR immediate byte to r/m byte
81 /6 iw	XOR r/m16,imm16	2/7	Exclusive-OR immediate word to r/m word
81 /6 id	XOR r/m32,imm32	2/7	Exclusive-OR immediate dword to r/m dword
83 /6 ib	XOR r/m16,imm8	2/7	XOR sign-extended immediate byte with r/m word
83 /6 ib	XOR r/m32,imm8	2/7	XOR sign-extended immediate byte with r/m dword
30 /r	XOR r/m8,r8	2/6	Exclusive-OR byte register to r/m byte
31 /r	XOR r/m16,r16	2/6	Exclusive-OR word register to r/m word
31 /r	XOR r/m32,r32	2/6	Exclusive-OR dword register to r/m dword
32 /r	XOR r8,r/m8	2/7	Exclusive-OR byte register to r/m byte
33 /r	XOR r16,r/m16	2/7	Exclusive-OR word register to r/m word
33 /r	XOR r32,r/m32	2/7	Exclusive-OR dword register to r/m dword

根据这个我们补充 make_EHelper(xor) 如下：

```
23  ✓ make_EHelper(xor) {
24      rtl_xor(&t2, &id_dest->val, &id_src->val);
25      operand_write(id_dest, &t2);
26
27      rtl_update_ZFSF(&t2, id_dest->width);
28      rtl_set_CF(&tzero);
29      rtl_set_OF(&tzero);
30
31      print_asm_template2(xor);
32  }
```

还有 opcode_table 和 make_group 如下：

```
/* 0x30 */ IDEXW(G2E,xor,1), IDEX(G2E,xor), IDEXW(E2G,xor,1),IDEX(E2G,xor),
/* 0x34 */ IDEXW(I2a,xor,1), IDEX(I2a,xor), EMPTY, EMPTY,
```

```
43  make_group(gp1,
44      EX(add), EX(or), EX adc), EX(sbb),
45      EX(and), EX(sub), EX(xor), EX(cmp))
```

1.5 补全 ret 指令

最后是 ret 指令，查阅 i386 手册得到：

RET — Return from Procedure

Opcode	Instruction	Clocks	Description
C3	RET	10+m	Return (near) to caller
CB	RET	18+m, pm=32+m	Return (far) to caller, same privilege
CB	RET	pm=68	Return (far), lesser privilege, switch stacks
C2 iw	RET imm16	10+m	Return (near), pop imm16 bytes of parameters
CA iw	RET imm16	18+m, pm=32+m	Return (far), same privilege, pop imm16 bytes
CA iw	RET imm16	pm=68	Return (far), lesser privilege, pop imm16 bytes

根据这个我们补充 make_EHelper(ret) 如下：

```
35  make_EHelper(ret) {
36      rtl_pop(&t0);
37      if (decoding.opcode == 0xc2) {
38          rtl_addi(&cpu.esp, &cpu.esp, id_dest->imm & 0xffff);
39      }
40      decoding.jump_eip = t0;
41      decoding.is_jump = 1;
42
43      print_asm("ret");
44  }
```

还有 opcode_table 如下：

```
124  /* 0xc0 */ IDEXW(gp2_Ib2E, gp2, 1), IDEX(gp2_Ib2E, gp2), IDEXW(I,ret,2), EX(ret),
```

到这为止，dummy 需要的指令就补充完成了，接下来测试运行一下 dummy：

```
SuYaoLei@988d7f56373a:~/code/ics2017/nexus-am/tests/cputest$ make ARCH=x86-nemu ALL=dummy run
Building dummy [x86-nemu]
Building am [x86-nemu]
make[2]: *** No targets specified and no makefile found. Stop.
[src/monitor/monitor.c,65,load_img] The image is /home/SuYaoLei/code/ics2017/nexus-am/tests/cputest/build/dummy-x86-nemu.bin
Welcome to NEMU!
[src/monitor/monitor.c,30,welcome] Build time: 10:17:44, Apr 22 2026
For help, type "help"
(nemu) c
nemu: HIT GOOD TRAP at eip = 0x00100055
(nemu) █
```

能看到程序运行成功，实现没有问题。

问题：立即数背后的故事

立即数背后的故事

在 `decode_op_l()` 函数中通过 `instr_fetch()` 函数获得指令中的立即数. 别看这里就这么一行代码, 其实背后隐藏着针对字节序的慎重考虑. 我们知道 x86 是小端机, 当你使用高级语言或者汇编语言写了一个 32 位常数 0x1234 的时候, 在生成的二进制代码中, 这个常数对应的字节序列如下 (假设这个常数在内存中的起始地址是 x):

```
x    x+1  x+2  x+3
+----+----+----+----+
| 34 | 12 | 00 | 00 |
+----+----+----+----+
```

而大多数 PC 机都是小端架构 (我们相信没有同学会使用 IBM 大型机来做 PA), 当 NEMU 运行的时候, `op_src->imm=instr_fetch(eip, 4)`, 这行代码会将 34 12 00 00 这个字节序列原封不动地从内存读入 `imm` 变量中, 主机的 CPU 会按照小端方式来解释这一字节序列, 于是会得到 0x1234, 符合我们的预期结果。

Motorola 68k 系列的处理器都是大端架构的. 现在问题来了, 考虑以下两种情况:

- ❖ 假设我们需要将 NEMU 运行在 Motorola 68k 的机器上 (把 NEMU 的源代码编译成 Motorola 68k 的机器码)
- ❖ 假设我们需要编写一个新的模拟器 NEMU-Motorola-68k, 模拟器本身运行在 x86 架构中, 但它模拟的是 Motorola 68k 程序的执行

在这两种情况下, 你需要注意些什么问题? 为什么会产生这些问题? 怎么解决它们? 事实上不仅仅是立即数的访问, 长度大于 1 字节的内存访问都需要考虑类似的问题. 我们在这里把问题统一抛出来, 以后就不再单独讨论了.

这两种情况本质上都是主机字节序和目标字节序不一致, 因此只需要在执行指令时, 都显式手动控制字节序, 比如可以写一个接口, 在执行指令时都要通过接口转换为正确的字节序。

2. 实现更多的指令

在 `nexus-am/tests/cputest/tests` 目录下有很多个类似于上面的 dummy 的测试用例, 要做的就是编译执行这些程序, 显而易见他们会报错, 因此要像上面实现 dummy 一样补全每个测试用例需要的指令。

2.1 补全 movzx 和 movsx 指令

从 i386 手册得到

MOVZX — Move with Zero-Extend

Opcode	Instruction	Clocks	Description
0F B6 /r	MOVZX r16, r/m8	3/6	Move byte to word with zero-extend
0F B6 /r	MOVZX r32, r/m8	3/6	Move byte to dword, zero-extend
0F B7 /r	MOVZX r32, r/m16	3/6	Move word to dword, zero-extend

MOVSX — Move with Sign-Extend

Opcode	Instruction	Clocks	Description
0F BE /r	MOVSX r16,r/m8	3/6	Move byte to word with sign-extend
0F BE /r	MOVSX r32,r/m8	3/6	Move byte to dword, sign-extend
0F BF /r	MOVSX r32,r/m16	3/6	Move word to dword, sign-extend

由于 make_EHelper(movzx) 和 make_EHelper(movsx) 已经实现，因此我们只需要对照补充 opcode_table 即可，如下：

188	/* 0xb4 */	EMPTY, EMPTY, IDEXW(mov_E2G, movzx, 1), IDEXW(mov_E2G, movzx, 2),
189	/* 0xb8 */	EMPTY, EMPTY, EMPTY, EMPTY,
190	/* 0xbc */	EMPTY, EMPTY, IDEXW(mov_E2G, movsx, 1), IDEXW(mov_E2G, movsx, 2),

2.2 补全 lea 指令

i386 手册内容如下：

LEA — Load Effective Address

Opcode	Instruction	Clocks	Description
8D /r	LEA r16,m	2	Store effective address for m in register r16
8D /r	LEA r32,m	2	Store effective address for m in register r32
8D /r	LEA r16,m	2	Store effective address for m in register r16
8D /r	LEA r32,m	2	Store effective address for m in register r32

同样的，make_EHelper(lea) 已经实现，因此只需要对照补充 opcode_table 即可，如下：

110	/* 0x8c */	EMPTY, IDEX(lea_M2G,lea), EMPTY, EMPTY,
-----	------------	---

2.3 补全 add 指令

同样 i386 手册内容如下：

ADD — Add

Opcode	Instruction	Clocks	Description
04 ib	ADD AL,imm8	2	Add immediate byte to AL
05 iw	ADD AX,imm16	2	Add immediate word to AX
05 id	ADD EAX,imm32	2	Add immediate dword to EAX
80 /0 ib	ADD r/m8,imm8	2/7	Add immediate byte to r/m byte
81 /0 iw	ADD r/m16,imm16	2/7	Add immediate word to r/m word
81 /0 id	ADD r/m32,imm32	2/7	Add immediate dword to r/m dword
83 /0 ib	ADD r/m16,imm8	2/7	Add sign-extended immediate byte to r/m word
83 /0 ib	ADD r/m32,imm8	2/7	Add sign-extended immediate byte to r/m dword
00 /r	ADD r/m8,r8	2/7	Add byte register to r/m byte
01 /r	ADD r/m16,r16	2/7	Add word register to r/m word
01 /r	ADD r/m32,r32	2/7	Add dword register to r/m dword
02 /r	ADD r8,r/m8	2/6	Add r/m byte to byte register
03 /r	ADD r16,r/m16	2/6	Add r/m word to word register
03 /r	ADD r32,r/m32	2/6	Add r/m dword to dword register

补充 make_EHelper(add) 如下：

```
3  make_EHelper(add) {
4      rtl_add(&t2, &id_dest->val, &id_src->val);
5      operand_write(id_dest, &t2);
6
7      rtl_update_ZFSF(&t2, id_dest->width);
8
9      rtl_sltu(&t0, &t2, &id_dest->val);
10     rtl_set_CF(&t0);
11
12     rtl_xor(&t0, &id_dest->val, &id_src->val);
13     rtl_not(&t0);
14     rtl_xor(&t1, &id_dest->val, &t2);
15     rtl_and(&t0, &t0, &t1);
16     rtl_msb(&t0, &t0, id_dest->width);
17     rtl_set_OF(&t0);
18
19     print_asm_template2(add);
20 }
```

opcode_table 如下：

```
75  /* 0x00 */  IDEXW(G2E, add, 1), IDEX(G2E, add), IDEXW(E2G, add, 1), IDEX(E2G, add),
76  /* 0x04 */  IDEXW(I2a, add, 1), IDEX(I2a, add), EMPTY, EMPTY,
```

make_group 如下：

```
43  make_group(gp1,
44      EX(add), EX(or), EX(adc), EX(sbb),
45      EX(and), EX(sub), EX(xor), EX(cmp))
```

2.4 补全 cmp 指令

i386 手册内容如下：

CMP — Compare Two Operands

Opcode	Instruction	Clocks	Description
3C ib	CMP AL,imm8	2	Compare immediate byte to AL
3D iw	CMP AX,imm16	2	Compare immediate word to AX
3D id	CMP EAX,imm32	2	Compare immediate dword to EAX
80 /7 ib	CMP r/m8,imm8	2/5	Compare immediate byte to r/m byte
81 /7 iw	CMP r/m16,imm16	2/5	Compare immediate word to r/m word
81 /7 id	CMP r/m32,imm32	2/5	Compare immediate dword to r/m dword
83 /7 ib	CMP r/m16,imm8	2/5	Compare sign extended immediate byte to r/m word
83 /7 id	CMP r/m32,imm8	2/5	Compare sign extended immediate byte to r/m dword
38 /r	CMP r/m8,r8	2/5	Compare byte register to r/m byte
39 /r	CMP r/m16,r16	2/5	Compare word register to r/m word
39 /r	CMP r/m32,r32	2/5	Compare dword register to r/m dword
3A /r	CMP r8,r/m8	2/6	Compare r/m byte to byte register
3B /r	CMP r16,r/m16	2/6	Compare r/m word to word register
3B /r	CMP r32,r/m32	2/6	Compare r/m dword to dword register

make_EHelper(cmp) 如下:

```
40  make_EHelper(cmp) {
41      rtl_sub(&t2, &id_dest->val, &id_src->val);
42
43      rtl_update_ZFSF(&t2, id_dest->width);
44
45      rtl_sltu(&t0, &id_dest->val, &id_src->val);
46      rtl_set_CF(&t0);
47
48      rtl_xor(&t0, &id_dest->val, &id_src->val);
49      rtl_xor(&t1, &id_dest->val, &t2);
50      rtl_and(&t0, &t0, &t1);
51      rtl_msb(&t0, &t0, id_dest->width);
52      rtl_set_OF(&t0);
53
54      print_asm_template2(cmp);
55 }
```

opcode_table 如下:

```
89  /* 0x38 */ IDEXW(G2E, cmp, 1), IDEX(G2E, cmp), IDEXW(E2G, cmp, 1), IDEX(E2G, cmp),
90  /* 0x3c */ IDEXW(I2a, cmp, 1), IDEX(I2a, cmp), EMPTY, EMPTY,
```

make_group 同上。

2.5 补全 inc 指令

i386 手册内容如下:

INC — Increment by 1

Opcode	Instruction	Clocks	Description
FE /0	INC r/m8		Increment r/m byte by 1
FF /0	INC r/m16		Increment r/m word by 1
FF /6	INC r/m32		Increment r/m dword by 1
40 + rw	INC r16		Increment word register by 1
40 + rd	INC r32		Increment dword register by 1

make_EHelper(inc) 和 opcode_table 如下：

```
57  make_EHelper(inc) {
58      rtl_addi(&t2, &id_dest->val, 1);
59      operand_write(id_dest, &t2);
60
61      rtl_update_ZFSF(&t2, id_dest->width);
62
63      rtl_li(&t0, 1);
64      rtl_xor(&t0, &id_dest->val, &t0);
65      rtl_not(&t0);
66      rtl_xor(&t1, &id_dest->val, &t2);
67      rtl_and(&t0, &t0, &t1);
68      rtl_msb(&t0, &t0, id_dest->width);
69      rtl_set_OF(&t0);
70
71      print_asm_template1(inc);
72 }
```

```
91  /* 0x40 */ IDEX(r, inc), IDEX(r, inc), IDEX(r, inc), IDEX(r, inc),
92  /* 0x44 */ IDEX(r, inc), IDEX(r, inc), IDEX(r, inc), IDEX(r, inc),
```

make_group 如下：

```
57  /* 0xfe */
58  ✓ make_group(gp4,
59  ✓   EX(inc), EX(dec), EMPTY, EMPTY,
60      EMPTY, EMPTY, EMPTY, EMPTY)
61
62  /* 0xff */
63  ✓ make_group(gp5,
64      EX(inc), EX(dec), EX(call_rm), EX(neg),
65      EX(jmp_rm), EMPTY, EX(push), EMPTY)
```

2.6 补全 dec 指令

i386 手册内容如下：

DEC — Decrement by 1

Opcode	Instruction	Clocks	Description
FE /1	DEC r/m8	2/6	Decrement r/m byte by 1
FF /1	DEC r/m16	2/6	Decrement r/m word by 1
	DEC r/m32	2/6	Decrement r/m dword by 1
48+rw	DEC r16	2	Decrement word register by 1
48+rw	DEC r32	2	Decrement dword register by 1

make_EHelper(dec) 和 opcode_table 如下：

```
74  make_EHelper(dec) {
75      rtl_subi(&t2, &id_dest->val, 1);
76      operand_write(id_dest, &t2);
77
78      rtl_update_ZFSF(&t2, id_dest->width);
79
80      rtl_li(&t0, 1);
81      rtl_xor(&t0, &id_dest->val, &t0);
82      rtl_xor(&t1, &id_dest->val, &t2);
83      rtl_and(&t0, &t0, &t1);
84      rtl_msb(&t0, &t0, id_dest->width);
85      rtl_set_OF(&t0);
86
87      print_asm_template1(dec);
88  }
```

```
93  /* 0x48 */ IDEX(r, dec), IDEX(r, dec), IDEX(r, dec), IDEX(r, dec),
94  /* 0x4c */ IDEX(r, dec), IDEX(r, dec), IDEX(r, dec), IDEX(r, dec),
```

make_group 如下：

```
57  /* 0xfe */
58  make_group(gp4,
59      EX(inc), EX(dec), EMPTY, EMPTY,
60      EMPTY, EMPTY, EMPTY, EMPTY)
61
62  /* 0xff */
63  make_group(gp5,
64      EX(inc), EX(dec), EX(call_rm), EX(neg),
65      EX(jmp_rm), EMPTY, EX(push), EMPTY)
```

2.7 补全 neg 指令

i386 手册内容如下：

NEG — Two's Complement Negation

Opcode	Instruction	Clocks	Description
F6	/3 NEG r/m8	2/6	Two's complement negate r/m byte
F7	/3 NEG r/m16	2/6	Two's complement negate r/m word
F7	/3 NEG r/m32	2/6	Two's complement negate r/m dword

Operation

IF $r/m = 0$ THEN $CF \leftarrow 0$ ELSE $CF \leftarrow 1$; FI;
 $r/m \leftarrow -r/m$;

make_EHelper(neg) 如下:

```
90  make_EHelper(neg) {
91      rtl_sub(&t2, &tzero, &id_dest->val);
92      operand_write(id_dest, &t2);
93
94      rtl_update_ZFSF(&t2, id_dest->width);
95
96      rtl_neq0(&t0, &id_dest->val);
97      rtl_set_CF(&t0);
98
99      rtl_xor(&t0, &tzero, &id_dest->val);
100     rtl_xor(&t1, &tzero, &t2);
101     rtl_and(&t0, &t0, &t1);
102     rtl_msb(&t0, &t0, id_dest->width);
103     rtl_set_OF(&t0);
104
105     print_asm_template1(neg);
106 }
```

make_group 如下:

```
52  /* 0xf6, 0xf7 */
53  make_group(gp3,
54      IDEX(test_I, test), EMPTY, EX(not), EX(neg),
55      EX(mul), EX(imul1), EX(div), EX(idiv))
```

```
62  /* 0xff */
63  make_group(gp5,
64      EX(inc), EX(dec), EX(call_rm), EX(neg),
65      EX(jmp_rm), EMPTY, EX(push), EMPTY)
```

2.8 补全 adc 指令

i386 手册内容如下:

ADC — Add with Carry

Opcode	Instruction	Clocks	Description
14 ib	ADC AL,imm8	2	Add with carry immediate byte to AL
15 iw	ADC AX,imm16	2	Add with carry immediate word to AX
15 id	ADC EAX,imm32	2	Add with carry immediate dword to EAX
80 /2 ib	ADC r/m8,imm8	2/7	Add with carry immediate byte to r/m byte
81 /2 iw	ADC r/m16,imm16	2/7	Add with carry immediate word to r/m word
81 /2 id	ADC r/m32,imm32	2/7	Add with CF immediate dword to r/m dword
83 /2 ib	ADC r/m16,imm8	2/7	Add with CF sign-extended immediate byte to r/m word
83 /2 ib	ADC r/m32,imm8	2/7	Add with CF sign-extended immediate byte into r/m dword
10 /r	ADC r/m8,r8	2/7	Add with carry byte register to r/m byte
11 /r	ADC r/m16,r16	2/7	Add with carry word register to r/m word
11 /r	ADC r/m32,r32	2/7	Add with CF dword register to r/m dword
12 /r	ADC r8,r/m8	2/6	Add with carry r/m byte to byte register
13 /r	ADC r16,r/m16	2/6	Add with carry r/m word to word register
13 /r	ADC r32,r/m32	2/6	Add with CF r/m dword to dword register

以及它的操作：

Operation

DEST ← DEST + SRC + CF;

opcode_table 和 make_group 如下：

```
79  /* 0x10 */ IDEXW(G2E, adc, 1), IDEX(G2E, adc), IDEXW(E2G, adc, 1), IDEX(E2G, adc),
80  /* 0x14 */ IDEXW(I2a, adc, 1), IDEX(I2a, adc), EMPTY, EMPTY,

42  /* 0x80, 0x81, 0x83 */
43  make_group(gp1,
44      EX(add), EX(or), EX(adc), EX(sbb),
45      EX(and), EX(sub), EX(xor), EX(cmp))
```

2.9 补全 sbb 指令

i386 手册内容如下：

SBB — Integer Subtraction with Borrow

Opcode	Instruction	Clocks	Description
1C ib	SBB AL,imm8	2	Subtract with borrow immediate byte from AL
1D iw	SBB AX,imm16	2	Subtract with borrow immediate word from AX
1D id	SBB EAX,imm32	2	Subtract with borrow immediate dword from EAX
80 /3 ib	SBB r/m8,imm8	2/7	Subtract with borrow immediate byte from r/m byte
81 /3 iw	SBB r/m16,imm16	2/7	Subtract with borrow immediate from r/m word
81 /3 id	SBB r/m32,imm32	2/7	Subtract with borrow immediate dword from r/m dword
83 /3 ib	SBB r/m16,imm8	2/7	Subtract with borrow sign-extended immediate byte from r/m word
83 /3 ib	SBB r/m32,imm8	2/7	Subtract with borrow sign-extended immediate byte from r/m dword
18 /r	SBB r/m8,r8	2/6	Subtract with borrow byte register from r/m byte
19 /r	SBB r/m16,r16	2/6	Subtract with borrow word register from r/m word
19 /r	SBB r/m32,r32	2/6	Subtract with borrow dword from r/m dword
1A /r	SBB r8,r/m8	2/7	Subtract with borrow byte register from r/m byte
1B /r	SBB r16,r/m16	2/7	Subtract with borrow word register from r/m word
1B /r	SBB r32,r/m32	2/7	Subtract with borrow dword register from r/m dword

op_code_table 和 make_group 如下：

```
81      /* 0x18 */ IDEXW(G2E, sbb, 1), IDEX(G2E, sbb), IDEXW(E2G, sbb, 1), IDEX(E2G, sbb),
82      /* 0x1c */ IDEXW(I2a, sbb, 1), IDEX(I2a, sbb), EMPTY, EMPTY,

42      /* 0x80, 0x81, 0x83 */
43      make_group(gp1,
44          EX(add), EX(or), EX(adc), EX(sbb),
45          EX(and), EX(sub), EX(xor), EX(cmp))
```

2.10 补全 mul 指令

i386 手册内容如下：

MUL — Unsigned Multiplication of AL or AX

Opcode	Instruction	Clocks	Description
F6 /4	MUL AL,r/m8	9-14/12-17	Unsigned multiply (AX ← AL * r/m byte)
F7 /4	MUL AX,r/m16	9-22/12-25	Unsigned multiply (DX:AX ← AX * r/m word)
F7 /4	MUL EAX,r/m32	9-38/12-41	Unsigned multiply (EDX:EAX ← EAX * r/m dword)

make_group 如下：

```
52      /* 0xf6, 0xf7 */
53      make_group(gp3,
54          IDEX(test_I, test), EMPTY, EX(not), EX(neg),
55          EX(mul), EX(imul1), EX(div), EX(idiv))
```

2.11 补全 imul 指令

根据 i386 手册：

IMUL — Signed Multiply

Opcode	Instruction	Clocks	Description
F6 /5	IMUL r/m8	9-14/12-17	AX← AL * r/m byte
F7 /5	IMUL r/m16	9-22/12-25	DX:AX ← AX * r/m word
F7 /5	IMUL r/m32	9-38/12-41	EDX:EAX ← EAX * r/m dword
0F AF /r	IMUL r16,r/m16	9-22/12-25	word register ← word register * r/m word
0F AF /r	IMUL r32,r/m32	9-38/12-41	dword register ← dword register * r/m dword
6B /r ib	IMUL r16,r/m16,imm8	9-14/12-17	word register ← r/m16 * sign-extended immediate byte
6B /r ib	IMUL r32,r/m32,imm8	9-14/12-17	dword register ← r/m32 * sign-extended immediate byte
6B /r ib	IMUL r16,imm8	9-14/12-17	word register ← word register * sign-extended immediate byte
6B /r ib	IMUL r32,imm8	9-14/12-17	dword register ← dword register * sign-extended immediate byte
69 /r iw	IMUL r16,r/m16,imm16	9-22/12-25	word register ← r/m16 * immediate word
69 /r id	IMUL r32,r/m32,imm32	9-38/12-41	dword register ← r/m32 * immediate dword
69 /r iw	IMUL r16,imm16	9-22/12-25	word register ← r/m16 * immediate word
69 /r id	IMUL r32,imm32	9-38/12-41	dword register ← r/m32 * immediate dword

imul 指令被分为1、2、3三种形式，分别表示单操作数、双操作数以及三个操作数的形式，实现如下：


```

52      /* 0xf6, 0xf7 */
53  ▾ make_group(gp3,
54      IDEX(test_I, test), EMPTY, EX(not), EX(neg),
55      EX(mul), EX(imul1), EX(div), EX(idiv))

101     /* 0x68 */ IDEX(I, push), IDEX(I_E2G, imul3), IDEX(push_SI, push), IDEX(SI_E2G, imul3),

186     /* 0xac */ EMPTY, EMPTY, EMPTY, IDEX(E2G, imul2),

```

2.12 补全 div 指令

i386 手册内容如下：

DIV — Unsigned Divide

Opcode	Instruction	Clocks	Description
F6 /6	DIV AL,r/m8	14/17	Unsigned divide AX by r/m byte (AL=Quo, AH=Rem)
F7 /6	DIV AX,r/m16	22/25	Unsigned divide DX:AX by r/m word (AX=Quo, DX=Rem)
F7 /6	DIV EAX,r/m32	38/41	Unsigned divide EDX:EAX by r/m dword (EAX=Quo, EDX=Rem)

make_group 如下：

```

52      /* 0xf6, 0xf7 */
53  make_group(gp3,
54      IDEX(test_I, test), EMPTY, EX(not), EX(neg),
55      EX(mul), EX(imul1), EX(div), EX(idiv))

```

2.13 补全 idiv 指令

i386 手册内容如下：

IDIV — Signed Divide

Opcode	Instruction	Clocks	Description
F6 /7	IDIV r/m8	19	Signed divide AX by r/m byte (AL=Quo, AH=Rem)
F7 /7	IDIV AX,r/m16	27	Signed divide DX:AX by EA word (AX=Quo, DX=Rem)
F7 /7	IDIV EAX,r/m32	43	Signed divide EDX:EAX by DWORD byte (EAX=Quo, EDX=Rem)

make_group 如下：

```

52      /* 0xf6, 0xf7 */
53  make_group(gp3,
54      IDEX(test_I, test), EMPTY, EX(not), EX(neg),
55      EX(mul), EX(imul1), EX(div), EX(idiv))

```

2.14 补全 and 指令

根据 i386 手册：

AND — Logical AND

Opcode	Instruction	Clocks	Description
24 ib	AND AL,imm8	2	AND immediate byte to AL
25 iw	AND AX,imm16	2	AND immediate word to AX
25 id	AND EAX,imm32	2	AND immediate dword to EAX
80 /4 ib	AND r/m8,imm8	2/7	AND immediate byte to r/m byte
81 /4 iw	AND r/m16,imm16	2/7	AND immediate word to r/m word
81 /4 id	AND r/m32,imm32	2/7	AND immediate dword to r/m dword
83 /4 ib	AND r/m16,imm8	2/7	AND sign-extended immediate byte with r/m word
83 /4 ib	AND r/m32,imm8	2/7	AND sign-extended immediate byte with r/m dword
20 /r	AND r/m8,r8	2/7	AND byte register to r/m byte
21 /r	AND r/m16,r16	2/7	AND word register to r/m word
21 /r	AND r/m32,r32	2/7	AND dword register to r/m dword
22 /r	AND r8,r/m8	2/6	AND r/m byte to byte register
23 /r	AND r16,r/m16	2/6	AND r/m word to word register
23 /r	AND r32,r/m32	2/6	AND r/m dword to dword register

Operation

DEST ← DEST AND SRC;

CF ← 0;

OF ← 0;

补充 make_EHelper(and) 如下:

```
12  v make_EHelper(and) {
13      rtl_and(&t2, &id_dest->val, &id_src->val);
14      operand_write(id_dest, &t2);
15
16      rtl_update_ZFSF(&t2, id_dest->width);
17      rtl_set_CF(&tzero);
18      rtl_set_OF(&tzero);
19
20      print_asm_template2(and);
21 }
```

make_group 和 opcode_table 如下:

```
42  /* 0x80, 0x81, 0x83 */
43  make_group(gp1,
44      EX(add), EX(or), EX adc), EX(sbb),
45      EX(and), EX(sub), EX(xor), EX(cmp))
```

```
83  /* 0x20 */ IDEXW(G2E, and, 1), ID EX(G2E, and), ID EXW(E2G, and, 1), ID EX(E2G, and),
84  /* 0x24 */ ID EXW(I2a, and, 1), ID EX(I2a, and), EMPTY, EMPTY,
```

2.15 补全 or 指令

根据 i386 手册:

OR — Logical Inclusive OR

Opcode	Instruction	Clocks	Description
0C ib	OR AL,imm8	2	OR immediate byte to AL
0D iw	OR AX,imm16	2	OR immediate word to AX
0D id	OR EAX,imm32	2	OR immediate dword to EAX
80 /1 ib	OR r/m8,imm8	2/7	OR immediate byte to r/m byte
81 /1 iw	OR r/m16,imm16	2/7	OR immediate word to r/m word
81 /1 id	OR r/m32,imm32	2/7	OR immediate dword to r/m dword
83 /1 ib	OR r/m16,imm8	2/7	OR sign-extended immediate byte with r/m word
83 /1 id	OR r/m32,imm8	2/7	OR sign-extended immediate byte with r/m dword
08 /r	OR r/m8,r8	2/6	OR byte register to r/m byte
09 /r	OR r/m16,r16	2/6	OR word register to r/m word
09 /r	OR r/m32,r32	2/6	OR dword register to r/m dword
0A /r	OR r8,r/m8	2/7	OR byte register to r/m byte
0B /r	OR r16,r/m16	2/7	OR word register to r/m word
0B /r	OR r32,r/m32	2/7	OR dword register to r/m dword

Operation

DEST ← DEST OR SRC;

CF ← 0;

OF ← 0

补充 make_EHelper(or) 如下:

```
34  ∨ make_EHelper(or) {
35      rtl_or(&t2, &id_dest->val, &id_src->val);
36      operand_write(id_dest, &t2);
37
38      rtl_update_ZFSF(&t2, id_dest->width);
39      rtl_set_CF(&tzero);
40      rtl_set_OF(&tzero);
41
42      print_asm_template2(or);
43  }
```

make_group 和 opcode_table 如下:

```
42  /* 0x80, 0x81, 0x83 */
43  make_group(gp1,
44      EX(add), EX(or), EX(adc), EX(sbb),
45      EX(and), EX(sub), EX(xor), EX(cmp))

77  /* 0x08 */ IDExW(G2E, or, 1), IDEx(G2E, or), IDExW(E2G, or, 1), IDEx(E2G, or),
78  /* 0x0c */ IDExW(I2a, or, 1), IDEx(I2a, or), EMPTY, EX(2byte_esc),
```

2.16 补全 test 指令

i386 手册内容如下:

TEST — Logical Compare

Opcode	Instruction	Clocks	Description
A8 ib	TEST AL,imm8	2	AND immediate byte with AL
A9 iw	TEST AX,imm16	2	AND immediate word with AX
A9 id	TEST EAX,imm32	2	AND immediate dword with EAX
F6 /0 ib	TEST r/m8,imm8	2/5	AND immediate byte with r/m byte
F7 /0 iw	TEST r/m16,imm16	2/5	AND immediate word with r/m word
F7 /0 id	TEST r/m32,imm32	2/5	AND immediate dword with r/m dword
84 /r	TEST r/m8,r8	2/5	AND byte register with r/m byte
85 /r	TEST r/m16,r16	2/5	AND word register with r/m word
85 /r	TEST r/m32,r32	2/5	AND dword register with r/m dword

Operation

DEST : = LeftSRC AND RightSRC;

CF ← 0;

OF ← 0;

make_EHelper(test) 如下:

```
3  √ make_EHelper(test) {
4      rtl_and(&t2, &id_dest->val, &id_src->val);
5      rtl_update_ZFSF(&t2, id_dest->width);
6      rtl_set_CF(&tzero);
7      rtl_set_OF(&tzero);
8
9      print_asm_template2(test);
10 }
```

make_group 和 opcode_table 如下:

```
52  /* 0xf6, 0xf7 */
53  make_group(gp3,
54      IDEX(test_I, test), EMPTY, EX(not), EX(neg),
55      EX(mul), EX(imul1), EX(div), EX(idiv))
```

```
108 /* 0x84 */ IDEXW(G2E, test, 1), IDEX(G2E, test), EMPTY, EMPTY,
```

```
118 /* 0xa8 */ IDEXW(I2a, test, 1), IDEX(I2a, test), EMPTY, EMPTY,
```

2.17 补全 not 指令

i386 手册内容如下:

NOT — One's Complement Negation

Opcode		Instruction	Clocks	Description
F6	/2	NOT r/m8	2/6	Reverse each bit of r/m byte
F7	/2	NOT r/m16	2/6	Reverse each bit of r/m word
F7	/2	NOT r/m32	2/6	Reverse each bit of r/m dword

Operation

$r/m \leftarrow \text{NOT } r/m;$

make_EHelper(not) 如下:

```
115  ∨ make_EHelper(not) {
116      rtl_mv(&t2, &id_dest->val);
117      rtl_not(&t2);
118      operand_write(id_dest, &t2);
119
120      print_asm_template1(not);
121  }
```

make_group 如下:

```
52      /* 0xf6, 0xf7 */
53      make_group(gp3,
54          IDEX(test_I, test), EMPTY, EX(not), EX(neg),
55          EX(mul), EX(imul1), EX(div), EX(idiv))
```

2.18 补全 sar、shl、shr 指令

这三条是算数移位和逻辑移位的指令，i386 手册如下:

SAL/SAR/SHL/SHR — Shift Instructions

Opcode		Instruction	Clocks	Description
D0	/4	SAL r/m8,1	3/7	Multiply r/m byte by 2, once
D2	/4	SAL r/m8,CL	3/7	Multiply r/m byte by 2, CL times
C0	/4 ib	SAL r/m8,imm8	3/7	Multiply r/m byte by 2, imm8 times
D1	/4	SAL r/m16,1	3/7	Multiply r/m word by 2, once
D3	/4	SAL r/m16,CL	3/7	Multiply r/m word by 2, CL times
C1	/4 ib	SAL r/m16,imm8	3/7	Multiply r/m word by 2, imm8 times
D1	/4	SAL r/m32,1	3/7	Multiply r/m dword by 2, once
D3	/4	SAL r/m32,CL	3/7	Multiply r/m dword by 2, CL times
C1	/4 ib	SAL r/m32,imm8	3/7	Multiply r/m dword by 2, imm8 times
D0	/7	SAR r/m8,1	3/7	Signed divide ⁽¹⁾ r/m byte by 2, once
D2	/7	SAR r/m8,CL	3/7	Signed divide ⁽¹⁾ r/m byte by 2, CL times
C0	/7 ib	SAR r/m8,imm8	3/7	Signed divide ⁽¹⁾ r/m byte by 2, imm8 times
D1	/7	SAR r/m16,1	3/7	Signed divide ⁽¹⁾ r/m word by 2, once
D3	/7	SAR r/m16,CL	3/7	Signed divide ⁽¹⁾ r/m word by 2, CL times
C1	/7 ib	SAR r/m16,imm8	3/7	Signed divide ⁽¹⁾ r/m word by 2, imm8 times
D1	/7	SAR r/m32,1	3/7	Signed divide ⁽¹⁾ r/m dword by 2, once
D3	/7	SAR r/m32,CL	3/7	Signed divide ⁽¹⁾ r/m dword by 2, CL times
C1	/7 ib	SAR r/m32,imm8	3/7	Signed divide ⁽¹⁾ r/m dword by 2, imm8 times
D0	/4	SHL r/m8,1	3/7	Multiply r/m byte by 2, once
D2	/4	SHL r/m8,CL	3/7	Multiply r/m byte by 2, CL times
C0	/4 ib	SHL r/m8,imm8	3/7	Multiply r/m byte by 2, imm8 times
D1	/4	SHL r/m16,1	3/7	Multiply r/m word by 2, once
D3	/4	SHL r/m16,CL	3/7	Multiply r/m word by 2, CL times
C1	/4 ib	SHL r/m16,imm8	3/7	Multiply r/m word by 2, imm8 times
D1	/4	SHL r/m32,1	3/7	Multiply r/m dword by 2, once
D3	/4	SHL r/m32,CL	3/7	Multiply r/m dword by 2, CL times
C1	/4 ib	SHL r/m32,imm8	3/7	Multiply r/m dword by 2, imm8 times
D0	/5	SHR r/m8,1	3/7	Unsigned divide r/m byte by 2, once
D2	/5	SHR r/m8,CL	3/7	Unsigned divide r/m byte by 2, CL times
C0	/5 ib	SHR r/m8,imm8	3/7	Unsigned divide r/m byte by 2, imm8 times
D1	/5	SHR r/m16,1	3/7	Unsigned divide r/m word by 2, once
D3	/5	SHR r/m16,CL	3/7	Unsigned divide r/m word by 2, CL times
C1	/5 ib	SHR r/m16,imm8	3/7	Unsigned divide r/m word by 2, imm8 times
D1	/5	SHR r/m32,1	3/7	Unsigned divide r/m dword by 2, once
D3	/5	SHR r/m32,CL	3/7	Unsigned divide r/m dword by 2, CL times
C1	/5 ib	SHR r/m32,imm8	3/7	Unsigned divide r/m dword by 2, imm8 times

三条指令的 make_EHelper() 如下：

```

45  make_EHelper(sar) {
46      rtl_andi(&t0, &id_src->val, 0x1f);
47      if (t0 == 0) {
48          print_asm_template2(sar);
49          return;
50      }
51      rtl_sar(&t2, &id_dest->val, &t0);
52      operand_write(id_dest, &t2);
53      rtl_update_ZFSF(&t2, id_dest->width);
54      // unnecessary to update CF and OF in NEMU
55
56      print_asm_template2(sar);
57  }

```

```

79  make_EHelper(shl) {
80      rtl_andi(&t0, &id_src->val, 0x1f);
81      if (t0 == 0) {
82          print_asm_template2(shl);
83          return;
84      }
85      rtl_shl(&t2, &id_dest->val, &t0);
86      operand_write(id_dest, &t2);
87      rtl_update_ZFSF(&t2, id_dest->width);
88      // unnecessary to update CF and OF in NEMU
89
90      print_asm_template2(shl);
91  }

```

```

93  make_EHelper(shr) {
94      rtl_andi(&t0, &id_src->val, 0x1f);
95      if (t0 == 0) {
96          print_asm_template2(shr);
97          return;
98      }
99      rtl_shr(&t2, &id_dest->val, &t0);
100     operand_write(id_dest, &t2);
101     rtl_update_ZFSF(&t2, id_dest->width);
102     // unnecessary to update CF and OF in NEMU
103
104     print_asm_template2(shr);
105 }

```

其 make_group 设置如下：

```

47  /* 0xc0, 0xc1, 0xd0, 0xd1, 0xd2, 0xd3 */
48  make_group(gp2,
49      EX(rol), EMPTY, EMPTY, EMPTY,
50      EX(shl), EX(shr), EMPTY, EX(sar))

```

2.19 补全 rol 指令

rol 是循环左移，在后面第三阶段的一个测试程序 microbench 会使用到，这里一起说明，i386 手册内容如下：

RCL/RCR/ROL/ROR — Rotate

Opcode		Instruction	Clocks	Description
D0	/2	RCL r/m8,1	9/10	Rotate 9 bits (CF,r/m byte) left once
D2	/2	RCL r/m8,CL	9/10	Rotate 9 bits (CF,r/m byte) left CL times
C0	/2 ib	RCL r/m8,imm8	9/10	Rotate 9 bits (CF,r/m byte) left imm8 times
D1	/2	RCL r/m16,1	9/10	Rotate 17 bits (CF,r/m word) left once
D3	/2	RCL r/m16,CL	9/10	Rotate 17 bits (CF,r/m word) left CL times
C1	/2 ib	RCL r/m16,imm8	9/10	Rotate 17 bits (CF,r/m word) left imm8 times
D1	/2	RCL r/m32,1	9/10	Rotate 33 bits (CF,r/m dword) left once
D3	/2	RCL r/m32,CL	9/10	Rotate 33 bits (CF,r/m dword) left CL times
C1	/2 ib	RCL r/m32,imm8	9/10	Rotate 33 bits (CF,r/m dword) left imm8 times
D0	/3	RCR r/m8,1	9/10	Rotate 9 bits (CF,r/m byte) right once
D2	/3	RCR r/m8,CL	9/10	Rotate 9 bits (CF,r/m byte) right CL times
C0	/3 ib	RCR r/m8,imm8	9/10	Rotate 9 bits (CF,r/m byte) right imm8 times
D1	/3	RCR r/m16,1	9/10	Rotate 17 bits (CF,r/m word) right once
D3	/3	RCR r/m16,CL	9/10	Rotate 17 bits (CF,r/m word) right CL times
C1	/3 ib	RCR r/m16,imm8	9/10	Rotate 17 bits (CF,r/m word) right imm8 times
D1	/3	RCR r/m32,1	9/10	Rotate 33 bits (CF,r/m dword) right once
D3	/3	RCR r/m32,CL	9/10	Rotate 33 bits (CF,r/m dword) right CL times
C1	/3 ib	RCR r/m32,imm8	9/10	Rotate 33 bits (CF,r/m dword) right imm8 times
D0	/0	ROL r/m8,1	3/7	Rotate 8 bits r/m byte left once
D2	/0	ROL r/m8,CL	3/7	Rotate 8 bits r/m byte left CL times
C0	/0 ib	ROL r/m8,imm8	3/7	Rotate 8 bits r/m byte left imm8 times
D1	/0	ROL r/m16,1	3/7	Rotate 16 bits r/m word left once
D3	/0	ROL r/m16,CL	3/7	Rotate 16 bits r/m word left CL times
C1	/0 ib	ROL r/m16,imm8	3/7	Rotate 16 bits r/m word left imm8 times
D1	/0	ROL r/m32,1	3/7	Rotate 32 bits r/m dword left once
D3	/0	ROL r/m32,CL	3/7	Rotate 32 bits r/m dword left CL times
C1	/0 ib	ROL r/m32,imm8	3/7	Rotate 32 bits r/m dword left imm8 times
D0	/1	ROR r/m8,1	3/7	Rotate 8 bits r/m byte right once
D2	/1	ROR r/m8,CL	3/7	Rotate 8 bits r/m byte right CL times
C0	/1 ib	ROR r/m8,imm8	3/7	Rotate 8 bits r/m word right imm8 times
D1	/1	ROR r/m16,1	3/7	Rotate 16 bits r/m word right once
D3	/1	ROR r/m16,CL	3/7	Rotate 16 bits r/m word right CL times
C1	/1 ib	ROR r/m16,imm8	3/7	Rotate 16 bits r/m word right imm8 times
D1	/1	ROR r/m32,1	3/7	Rotate 32 bits r/m dword right once
D3	/1	ROR r/m32,CL	3/7	Rotate 32 bits r/m dword right CL times
C1	/1 ib	ROR r/m32,imm8	3/7	Rotate 32 bits r/m dword right imm8 times

make_EHelper(rol) 如下:

```

59  ~ make_EHelper(rol) {
60      rtl_andi(&t0, &id_src->val, 0x1f);
61
62      uint32_t bits = id_dest->width * 8;
63      uint32_t n = t0 % bits;
64  ~  if (n == 0) {
65      |      print_asm_template2(rol);
66      |      return;
67      |  }
68
69      uint32_t mask = (bits == 32 ? 0xffffffffu : ((1u << bits) - 1));
70      uint32_t val = id_dest->val & mask;
71      uint32_t res = ((val << n) | (val >> (bits - n))) & mask;
72
73      rtl_li(&t2, res);
74      operand_write(id_dest, &t2);
75
76      print_asm_template2(rol);
77  }

```

make_group 如下：

```
47  /* 0xc0, 0xc1, 0xd0, 0xd1, 0xd2, 0xd3 */
48  make_group(gp2,
49  EX(rol), EMPTY, EMPTY, EMPTY,
50  EX(shl), EX(shr), EMPTY, EX(sar))
```

2.20 补全 setcc 指令

setcc 是根据条件码来设置结果的指令，其实包含了很多指令，i386 手册内容如下：

SETcc — Byte Set on Condition

Opcode	Instruction	Clocks	Description
0F 97	SETA r/m8	4/5	Set byte if above (CF=0 and ZF=0)
0F 93	SETAE r/m8	4/5	Set byte if above or equal (CF=0)
0F 92	SETB r/m8	4/5	Set byte if below (CF=1)
0F 96	SETBE r/m8	4/5	Set byte if below or equal (CF=1 or (ZF=1)
0F 92	SETC r/m8	4/5	Set if carry (CF=1)
0F 94	SETE r/m8	4/5	Set byte if equal (ZF=1)
0F 9F	SETG r/m8	4/5	Set byte if greater (ZF=0 or SF=OF)
0F 9D	SETGE r/m8	4/5	Set byte if greater or equal (SF=OF)
0F 9C	SETL r/m8	4/5	Set byte if less (SF≠OF)
0F 9E	SETLE r/m8	4/5	Set byte if less or equal (ZF=1 and SF≠OF)
0F 96	SETNA r/m8	4/5	Set byte if not above (CF=1)
0F 92	SETNAE r/m8	4/5	Set byte if not above or equal (CF=1)
0F 93	SETNB r/m8	4/5	Set byte if not below (CF=0)
0F 97	SETNBE r/m8	4/5	Set byte if not below or equal (CF=0 and ZF=0)
0F 93	SETNC r/m8	4/5	Set byte if not carry (CF=0)
0F 95	SETNE r/m8	4/5	Set byte if not equal (ZF=0)
0F 9E	SETNG r/m8	4/5	Set byte if not greater (ZF=1 or SF≠OF)
0F 9C	SETNGE r/m8	4/5	Set if not greater or equal (SF≠OF)
0F 9D	SETNL r/m8	4/5	Set byte if not less (SF=OF)
0F 9F	SETNLE r/m8	4/5	Set byte if not less or equal (ZF=1 and SF≠OF)
0F 91	SETNO r/m8	4/5	Set byte if not overflow (OF=0)
0F 9B	SETNP r/m8	4/5	Set byte if not parity (PF=0)
0F 99	SETNS r/m8	4/5	Set byte if not sign (SF=0)
0F 95	SETNZ r/m8	4/5	Set byte if not zero (ZF=0)
0F 90	SETO r/m8	4/5	Set byte if overflow (OF=1)
0F 9A	SETP r/m8	4/5	Set byte if parity (PF=1)
0F 9A	SETPE r/m8	4/5	Set byte if parity even (PF=1)
0F 9B	SETPO r/m8	4/5	Set byte if parity odd (PF=0)
0F 98	SETS r/m8	4/5	Set byte if sign (SF=1)
0F 94	SETZ r/m8	4/5	Set byte if zero (ZF=1)

Operation

IF condition THEN r/m8 ← 1 ELSE r/m8 ← 0; FI;

这些指令的 opcode 是连续的一段区间（0x90–0x9F），每个 opcode 只是在条件码上不同，但整体行为完全一致。

具体来说，这 16 个 opcode 分别对应不同条件（如 sete、setne、setg 等），但它们的执行逻辑是统一的：根据某个条件是否成立，把目标操作数置为 0 或 1。因此在实现时，没有必要为每一个条件写一套独立逻辑，而是统一使用同一个执行函数 setcc，具体实现如下：

```

179 /* 0x90 */ IDEXW(gp7_E, setcc, 1), IDEXW(gp7_E, setcc, 1), IDEXW(gp7_E, setcc, 1), IDEXW(gp7_E, setcc, 1),
180 /* 0x94 */ IDEXW(gp7_E, setcc, 1), IDEXW(gp7_E, setcc, 1), IDEXW(gp7_E, setcc, 1), IDEXW(gp7_E, setcc, 1),
181 /* 0x98 */ IDEXW(gp7_E, setcc, 1), IDEXW(gp7_E, setcc, 1), IDEXW(gp7_E, setcc, 1), IDEXW(gp7_E, setcc, 1),
182 /* 0x9c */ IDEXW(gp7_E, setcc, 1), IDEXW(gp7_E, setcc, 1), IDEXW(gp7_E, setcc, 1), IDEXW(gp7_E, setcc, 1),

```

当然，还需要补全位于 nemu/src/cpu/exec/cc.c 的 void rtl_setcc() 函数：

```

5 void rtl_setcc(rtlreg_t* dest, uint8_t subcode) {
6     bool invert = subcode & 0x1;
7     enum {
8         CC_O, CC_NO, CC_B,  CC_NB,
9         CC_E, CC_NE, CC_BE, CC_NBE,
10        CC_S, CC_NS, CC_P,  CC_NP,
11        CC_L, CC_NL, CC_LE, CC_NLE
12    };
13
14    // TODO: Query EFLAGS to determine whether the condition code is satisfied.
15    // dest <- ( cc is satisfied ? 1 : 0)
16    switch (subcode & 0xe) {
17        case CC_O:
18            rtl_get_OF(dest);
19            break;
20        case CC_B:
21            rtl_get_CF(dest);
22            break;
23        case CC_E:
24            rtl_get_ZF(dest);
25            break;
26        case CC_BE:
27            rtl_get_CF(dest);
28            rtl_get_ZF(&t0);
29            rtl_or(dest, dest, &t0);
30            break;
31        case CC_S:
32            rtl_get_SF(dest);
33            break;
34        case CC_L:
35            rtl_get_SF(dest);
36            rtl_get_OF(&t0);
37            rtl_xor(dest, dest, &t0);
38            break;
39        case CC_LE:
40            rtl_get_SF(dest);
41            rtl_get_OF(&t0);
42            rtl_xor(dest, dest, &t0);
43            rtl_get_ZF(&t0);
44            rtl_or(dest, dest, &t0);
45            break;
46        default: panic("should not reach here");
47        case CC_P: panic("n86 does not have PF");
48    }

```

```

49
50  ∨  if (invert) {
51      rtl_xori(dest, dest, 0x1);
52  }
53  }

```

2.21 补全 call_rm 指令

call 是直接调用，目标地址在译码阶段就已经从指令中解析出来，并保存在 decoding.jump_eip 中，因此执行阶段只需要压入返回地址，然后根据这个已经计算好的目标地址完成跳转。

而 call_rm 是间接调用 (indirect call)，目标地址不是写在指令里的，而是来自寄存器或内存 (即 r/m)。因此在执行阶段需要从 id_dest->val 中读取真实的跳转地址，并手动赋值给 decoding.jump_eip，再进行跳转。

根据这个补全 make_EHelper(call_rm) 如下：

```

46  ∨  make_EHelper(call_rm) {
47      rtl_li(&t0, *eip);
48      rtl_push(&t0);
49      decoding.jump_eip = id_dest->val;
50      decoding.is_jump = 1;
51
52      print_asm("call %s", id_dest->str);
53  }

```

make_group 如下：

```

62      /* 0xff */
63  ∨  make_group(gp5,
64      EX(inc), EX(dec), EX(call_rm), EX(neg),
65      EX(jmp_rm), EMPTY, EX(push), EMPTY)

```

2.22 补全 jmp、jmp_rm 指令

原理同call，一个是直接跳转，一个是间接跳转。

看 i386 手册如下：

JMP — Jump

Opcode	Instruction	Clocks	Description
EB cb	JMP rel8	7+m	Jump short
E9 cw	JMP rel16	7+m	Jump near, displacement relative to next instruction
FF /4	JMP r/m16	7+m/10+m	Jump near indirect
EA cd	JMP ptr16:16	12+m,pm=27+m	Jump intersegment, 4-byte immediate address
EA cd	JMP ptr16:16	pm=45+m	Jump to call gate, same privilege
EA cd	JMP ptr16:16	ts	Jump via task state segment
EA cd	JMP ptr16:16	ts	Jump via task gate
FF /5	JMP m16:16	43+m,pm=31+m	Jump r/m16:16 indirect and intersegment
FF /5	JMP m16:16	pm=49+m	Jump to call gate, same privilege
FF /5	JMP m16:16	5 + ts	Jump via task state segment
FF /5	JMP m16:16	5 + ts	Jump via task gate
E9 cd	JMP rel32	7+m	Jump near, displacement relative to next instruction
FF /4	JMP r/m32	7+m,10+m	Jump near, indirect
EA cp	JMP ptr16:32	12+m,pm=27+m	Jump intersegment, 6-byte immediate address
EA cp	JMP ptr16:32	pm=45+m	Jump to call gate, same privilege
EA cp	JMP ptr16:32	ts	Jump via task state segment
EA cp	JMP ptr16:32	ts	Jump via task gate
FF /5	JMP m16:32	43+m,pm=31+m	Jump intersegment, address at r/m dword
FF /5	JMP m16:32	pm=49+m	Jump to call gate, same privilege
FF /5	JMP m16:32	5 + ts	Jump via task state segment
FF /5	JMP m16:32	5 + ts	Jump via task gate

make_group 和 opcode_table 如下：

```
62      /* 0xff */
63  √ make_group(gp5,
64      EX(inc), EX(dec), EX(call_rm), EX(neg),
65      EX(jmp_rm), EMPTY, EX(push), EMPTY)

134     /* 0xe8 */ IDEX(J,call), IDEX(J,jmp), IDEX(I,jmp_rm), IDEXW(J,jmp,1),
```

2.23 补全 jcc 指令

jcc 是条件跳转指令，有两种不同的编码格式，分别对应短跳转和近跳转，短跳转使用 8 位立即数偏移 (imm8)，并且需要做符号扩展，跳转范围较小 (-128 ~ +127)，而如果要跳转更远则需要使用近跳转。

其 opcode_table 如下：

```
103     /* 0x70 */ IDEXW(J,jcc,1), IDEXW(J,jcc,1), IDEXW(J,jcc,1), IDEXW(J,jcc,1),
104     /* 0x74 */ IDEXW(J,jcc,1), IDEXW(J,jcc,1), IDEXW(J,jcc,1), IDEXW(J,jcc,1),
105     /* 0x78 */ IDEXW(J,jcc,1), IDEXW(J,jcc,1), IDEXW(J,jcc,1), IDEXW(J,jcc,1),
106     /* 0x7c */ IDEXW(J,jcc,1), IDEXW(J,jcc,1), IDEXW(J,jcc,1), IDEXW(J,jcc,1),

175     /* 0x80 */ IDEX(J,jcc), IDEX(J,jcc), IDEX(J,jcc), IDEX(J,jcc),
176     /* 0x84 */ IDEX(J,jcc), IDEX(J,jcc), IDEX(J,jcc), IDEX(J,jcc),
177     /* 0x88 */ IDEX(J,jcc), IDEX(J,jcc), IDEX(J,jcc), IDEX(J,jcc),
178     /* 0x8c */ IDEX(J,jcc), IDEX(J,jcc), IDEX(J,jcc), IDEX(J,jcc),
```

2.24 补全 cwtl、cltd 指令

cwtl 是将较小位宽的数据符号扩展到更大位宽，而 cltd 是将 EAX 的符号扩展到 EDX:EAX，根据这个补全 make_EHelper() 如下：

```

58  make_EHelper(cwtl) {
59      if (decoding.is_operand_size_16) {
60          rtl_lr_b(&t0, R_AL);
61          rtl_sext(&t1, &t0, 1);
62          rtl_sr_w(R_AX, &t1);
63      }
64      else {
65          rtl_lr_w(&t0, R_AX);
66          rtl_sext(&t1, &t0, 2);
67          rtl_sr_l(R_EAX, &t1);
68      }
69
70      print_asm(decoding.is_operand_size_16 ? "cbtw" : "cwtl");
71  }

```

```

41  ∨ make_EHelper(cltd) {
42  ∨      if (decoding.is_operand_size_16) {
43          rtl_lr_w(&t0, R_AX);
44          rtl_msb(&t1, &t0, 2);
45          rtl_sub(&t1, &tzero, &t1);
46          rtl_sr_w(R_DX, &t1);
47      }
48  ∨      else {
49          rtl_lr_l(&t0, R_EAX);
50          rtl_msb(&t1, &t0, 4);
51          rtl_sub(&t1, &tzero, &t1);
52          rtl_sr_l(R_EDX, &t1);
53      }
54
55      print_asm(decoding.is_operand_size_16 ? "cwtl" : "cltd");
56  }

```

它们的 opcode_table 如下:

```

114  /* 0x98 */ EX(cwtl), EX(cltd), EMPTY, EMPTY,

```

2.25 补全 leave 指令

leave 指令是函数结束指令，用于恢复栈帧，相当于 mov esp, ebp 和 pop ebp，根据 i386 手册：

LEAVE — High Level Procedure Exit

Opcode	Instruction	Clocks	Description
C9	LEAVE	4	Set SP to BP, then pop BP
C9	LEAVE	4	Set ESP to EBP, then pop EBP

Operation

```
IF StackAddrSize = 16
THEN
    SP ← BP;
ELSE (* StackAddrSize = 32 *)
    ESP ← EBP;
FI;
IF OperandSize = 16
THEN
    BP ← Pop();
ELSE (* OperandSize = 32 *)
    EBP ← Pop();
FI;
```

make_EHelper(leave) 如下:

```
33  make_EHelper(leave) {
34      rtl_mv(&cpu.esp, &cpu.ebp);
35      rtl_pop(&t0);
36      rtl_sr_l(R_EBP, &t0);
37
38      print_asm("leave");
39  }
```

opcode_table 如下:

```
126  /* 0xc8 */  EMPTY, EX(leave), EMPTY, EMPTY,
```

2.26 补全 nop 指令

对于 nop 指令, 其实什么也不需要做, 在执行 add.c 程序的时候会出现如下的错误:

```
SuYaoLei@988d7f56373a:~/code/ics2017/nexus-am/tests/cputest$ make ARCH=x86-nemu ALL=add run
+ CC src/device/timer.c
+ LD build/nemu
[src/monitor/monitor.c,65,load_img] The image is /home/SuYaoLei/code/ics2017/nexus-am/tests/cputest/build/add-x86-nemu.bin
error: XDG_RUNTIME_DIR is invalid or not set in the environment.
Welcome to NEMU!
[src/monitor/monitor.c,30,welcome] Build time: 18:00:57, Apr 22 2026
For help, type "help"
(nemu) C
Unknown command 'C'
(nemu) c
invalid opcode(eip = 0x001000a6): 66 90 83 ec 0c 8b 04 b5 ...

There are two cases which will trigger this unexpected exception:
1. The instruction at eip = 0x001000a6 is not implemented.
2. Something is implemented incorrectly.
Find this eip(0x001000a6) in the disassembling result to distinguish which case it is.
```


程序在 0x1000a6 处出现错误，观察反汇编的文件，发现是这一行有问题：

```
88      1000a6: 66 90          xchg    %ax,%ax
```

但是 `xchg %ax, %ax` 的作用是交换寄存器 AX 与自身的值，但因为是同一个寄存器，所以实际什么都不会改变，相当于一空操作（NOP）指令，因此我们只需要在 op 为 90 的地方填上一条 `nop` 即可：

```
111     /* 0x90 */ EX(nop), EMPTY, EMPTY, EMPTY,
```

2.27 实现 differential testing

在 `nemu/src/monitor/diff-test/diff-test.c` 的 `difftest_step()` 函数中，我们需要把 NEMU 的 8 个通用寄存器和 `eip` 与从 QEMU 中读出的寄存器的值进行比较，如果发现值不一样，就输出相应的提示信息，并将 `diff` 标志设置为 `true`，在 `difftest_step()` 的最后，如果检测到 `diff` 标志为 `true`，就停止客户程序的运行。

具体代码实现如下：

```
151     // TODO: Check the registers state with QEMU.
152     // Set `diff` as `true` if they are not the same.
153     diff = diff || (r.eax != cpu.eax);
154     diff = diff || (r.ecx != cpu.ecx);
155     diff = diff || (r.edx != cpu.edx);
156     diff = diff || (r.ebx != cpu.ebx);
157     diff = diff || (r.esp != cpu.esp);
158     diff = diff || (r.ebp != cpu.ebp);
159     diff = diff || (r.esi != cpu.esi);
160     diff = diff || (r.edi != cpu.edi);
161     diff = diff || (r.eip != cpu.eip);
162
163     if (diff) {
164         printf("Differential test failed at eip = 0x%08x\n", eip);
165         printf("  eax: nemu=0x%08x qemu=0x%08x\n", cpu.eax, r.eax);
166         printf("  ecx: nemu=0x%08x qemu=0x%08x\n", cpu.ecx, r.ecx);
167         printf("  edx: nemu=0x%08x qemu=0x%08x\n", cpu.edx, r.edx);
168         printf("  ebx: nemu=0x%08x qemu=0x%08x\n", cpu.ebx, r.ebx);
169         printf("  esp: nemu=0x%08x qemu=0x%08x\n", cpu.esp, r.esp);
170         printf("  ebp: nemu=0x%08x qemu=0x%08x\n", cpu.ebp, r.ebp);
171         printf("  esi: nemu=0x%08x qemu=0x%08x\n", cpu.esi, r.esi);
172         printf("  edi: nemu=0x%08x qemu=0x%08x\n", cpu.edi, r.edi);
173         printf("  eip: nemu=0x%08x qemu=0x%08x\n", cpu.eip, r.eip);
174         printf("  eflags: nemu=0x%08x qemu=0x%08x\n", cpu.eflags, r.eflags);
175     }
```

之后打开 `nemu/include/common.h` 的 `DIFF_TEST`：

```
4     // #define DEBUG
5     #define DIFF_TEST
```

尝试运行 `add.c` 程序，能看到显示 “Connect to QEMU successfully”：

```

❖ SuYaolei@988d7f56373a:~/code/ics2017/nexus-am/tests/cputest$ make ARCH=x86-nemu ALL=add run
Building add [x86-nemu]
Building am [x86-nemu]
make[2]: *** No targets specified and no makefile found. Stop.
ld: warning: /home/SuYaolei/code/ics2017/nexus-am/tests/cputest/build/add-x86-nemu has a LOAD segment with RWX permissions
[src/monitor/diff-test/diff-test.c,97,init_difftest] Connect to QEMU successfully
[src/monitor/monitor.c,65,load_img] The image is /home/SuYaolei/code/ics2017/nexus-am/tests/cputest/build/add-x86-nemu.bin
Welcome to NEMU!
[src/monitor/monitor.c,30,welcome] Build time: 18:37:42, Apr 22 2026
For help, type "help"
(nemu) c
nemu: HIT GOOD TRAP at eip = 0x00100061
(nemu) █

```

2.28 回归测试

接下来根据指导书进行回归测试，在nemu/ 目录下运行

```
bash runall.sh
```

得到结果如下：

```

❖ SuYaolei@988d7f56373a:~/code/ics2017/nemu$ bash runall.sh
NEMU compile OK
compiling testcases...
testcases compile OK
[ add-longlong] PASS!
[ add] PASS!
[ bit] PASS!
[ bubble-sort] PASS!
[ dummy] PASS!
[ fact] PASS!
[ fib] PASS!
[ goldbach] PASS!
[ hello-str] PASS!
[ if-else] PASS!
[ leap-year] PASS!
[ load-store] PASS!
[ matrix-mul] PASS!
[ max] PASS!
[ min3] PASS!
[ mov-c] PASS!
[ movsx] PASS!
[ mul-longlong] PASS!
[ pascal] PASS!
[ prime] PASS!
[ quick-sort] PASS!
[ recursion] PASS!
[ select-sort] PASS!
[ shift] PASS!
[ shuixianhua] PASS!
[ string] PASS!
[ sub-longlong] PASS!
[ sum] PASS!
[ switch] PASS!
[ to-lower-case] PASS!
[ unalign] PASS!
[ wanshu] PASS!
❖ SuYaolei@988d7f56373a:~/code/ics2017/nemu$ █

```

这说明实现是正确的。

问题：堆和栈在哪里？

我们知道代码和数据都在可执行文件里面,但却没有提到(heap)和栈(stack).为什么堆和栈的内容没有放入可执行文件里面?那程序运行时用到的堆和栈又是怎么来的?

因为这两者在程序运行前是无法确定的。

栈用于函数调用过程中的局部变量、返回地址等，它的内容完全依赖程序运行时的调用路径和执行过程；堆（heap）用于动态内存分配，大小和使用情况取决于程序运行时的行为,这些内容在编译或链接阶段都是未知的，因此不可能预先放进可执行文件中。

程序运行时，操作系统在加载可执行文件时，会为进程分配一块虚拟地址空间，并在其中划分出栈和堆区域。栈通常由系统初始化（设置栈顶指针），随着函数调用自动增长/收缩；堆则从一块初始区域开始，通过系统调用按需扩展，由内存分配器进行管理。

因此，堆和栈是运行时动态生成和管理的内存区域，而不是静态程序内容，因此不会出现在可执行文件中。

问题：L语言的三种指令如下，假设除了输入变量之外,其它变量的初值都是0,并且假设程序执行到最后一条指令就结束,你可以仅用这三种指令写一个计算两个正整数相加的程序吗？

```
V = V + 1
V = V - 1
IF V != 0 GOTO LABEL
```

可以实现，整体思路就是把其中一个数当作计数器，不断把它减 1，同时把另一个数加 1，直到计数器变成 0，这样结果就累加完成了。

设两个输入正整数分别存放在 A 和 B 中，最终结果放在 A 中，可以写成类似如下的 .L 程序：

```
LOOP:
IF B != 0 GOTO BODY
GOTO END

BODY:
B = B - 1
A = A + 1
IF B != 0 GOTO LOOP

END:
```

问题：NEMU除了作为模拟器之外,还具有简单的调试功能,可以设置断点,查看程序状态.如果让你为NEMU添加如下功能你觉得应该如何实现?

当用户程序陷入死循环时,让用户程序暂停下来,并输出相应的提示信息

可以在 NEMU 中为指令执行增加一个计数器，每执行一条指令就累加一次，如果在没有触发程序结束或系统调用等情况下，指令执行次数超过某个设定阈值，就可以认为程序可能陷入了死循环，从而主动暂停执行。

进一步可以通过观察程序计数器（PC）的变化来提高判断的准确性。例如记录最近一段时间内的 PC 值，如果发现 PC 在少数几个地址之间反复跳转，或者长时间停留在同一位置，就说明程序很可能卡在一个循环中。

3.运行Hello World

根据指导书内容，需要先实现 in 和 out 指令，基本内容同上。

i386 手册描述如下：

IN — Input from Port

Opcode	Instruction	Clocks	Description
E4 ib	IN AL,imm8	12,pm=6*/26**	Input byte from immediate port into AL
E5 ib	IN AX,imm8	12,pm=6*/26**	Input word from immediate port into AX
E5 ib	IN EAX,imm8	12,pm=6*/26**	Input dword from immediate port into EAX
EC	IN AL,DX	13,pm=7*/27**	Input byte from port DX into AL
ED	IN AX,DX	13,pm=7*/27**	Input word from port DX into AX
ED	IN EAX,DX	13,pm=7*/27**	Input dword from port DX into EAX

NOTES:

- *If $CPL \leq IOPL$
- **If $CPL > IOPL$ or if in virtual 8086 mode

Operation

```
IF (PE = 1) AND ((VM = 1) OR (CPL > IOPL))
THEN (* Virtual 8086 mode, or protected mode with CPL > IOPL *)
  IF NOT I-O-Permission (SRC, width(SRC))
  THEN #GP(0);
  FI;
FI;
DEST ← [SRC]; (* Reads from I/O address space *)
```

OUT — Output to Port

Opcode	Instruction	Clocks	Description
E6 ib	OUT imm8,AL	10,pm=4*/24**	Output byte AL to immediate port number
E7 ib	OUT imm8,AX	10,pm=4*/24**	Output word AL to immediate port number
E7 ib	OUT imm8,EAX	10,pm=4*/24**	Output dword AL to immediate port number
EE	OUT DX,AL	11,pm=5*/25**	Output byte AL to port number in DX
EF	OUT DX,AX	11,pm=5*/25**	Output word AL to port number in DX
EF	OUT DX,EAX	11,pm=5*/25**	Output dword AL to port number in DX

NOTES:

- *If $CPL \leq IOPL$
- **If $CPL > IOPL$ or if in virtual 8086 mode

Operation

```
IF (PE = 1) AND ((VM = 1) OR (CPL > IOPL))
THEN (* Virtual 8086 mode, or protected mode with CPL > IOPL *)
    IF NOT I-O-Permission (DEST, width(DEST))
    THEN #GP(0);
    FI;
FI;
[DEST] ← SRC; (* I/O address space used *)
```

在 nemu/src/cpu/exec/system.c 里实现 make_EHelper(in) 和 make_EHelper(out) 如下:

```
47  make_EHelper(in) {
48      rtl_li(&t0, pio_read(id_src->val & 0xffff, id_dest->width));
49      operand_write(id_dest, &t0);
50
51      print_asm_template2(in);
52
53  #ifdef DIFF_TEST
54      diff_test_skip_qemu();
55  #endif
56  }
57
58  make_EHelper(out) {
59      pio_write(id_dest->val & 0xffff, id_src->width, id_src->val);
60
61      print_asm_template2(out);
62
63  #ifdef DIFF_TEST
64      diff_test_skip_qemu();
65  #endif
66  }
```

同时, 补充 opcode_table 如下:

```
133  /* 0xe4 */ IDEXW(in_I2a, in, 1), IDEX(in_I2a, in), IDEXW(out_a2I, out, 1), IDEX(out_a2I, out),
135  /* 0xec */ IDEXW(in_dx2a, in, 1), IDEX(in_dx2a, in), IDEXW(out_a2dx, out, 1), IDEX(out_a2dx, out),
```

实现后, 在 nexus-am/am/arch/x86-nemu/src/trm.c 中定义宏 HAS_SERIAL:

```
4 // Define this macro after serial has been implemented
5 #define HAS_SERIAL
```

然后在 nexus-am/apps/hello 目录下键入

```
make run
```

运行结果如下：

```
SuYaolei@988d7f56373a:~/code/ics2017/nexus-am/apps/hello$ make run
./build/nemu -l /home/SuYaolei/code/ics2017/nexus-am/apps/hello/build/nemu-log.txt /home/SuYaolei/code/ics2017/
llo-x86-nemu.bin
[src/monitor/diff-test/diff-test.c,97,init_difftest] Connect to QEMU successfully
[src/monitor/monitor.c,65,load_img] The image is /home/SuYaolei/code/ics2017/nexus-am/apps/hello/build/hello-x8
Welcome to NEMU!
[src/monitor/monitor.c,30,welcome] Build time: 19:40:00, Apr 23 2026
For help, type "help"
(nemu) c
Hello World!
Hello World!
Hello World!
Hello World!
Hello World!
Hello World!
Hello World!
Hello World!
Hello World!
Hello World!
Hello World!
nemu: HIT GOOD TRAP at eip = 0x00100071
(nemu) █
```

能看到成功输出了10行“Hello, world!”，实现是正确的。

问题：理解volatile关键字

理解volatile关键字

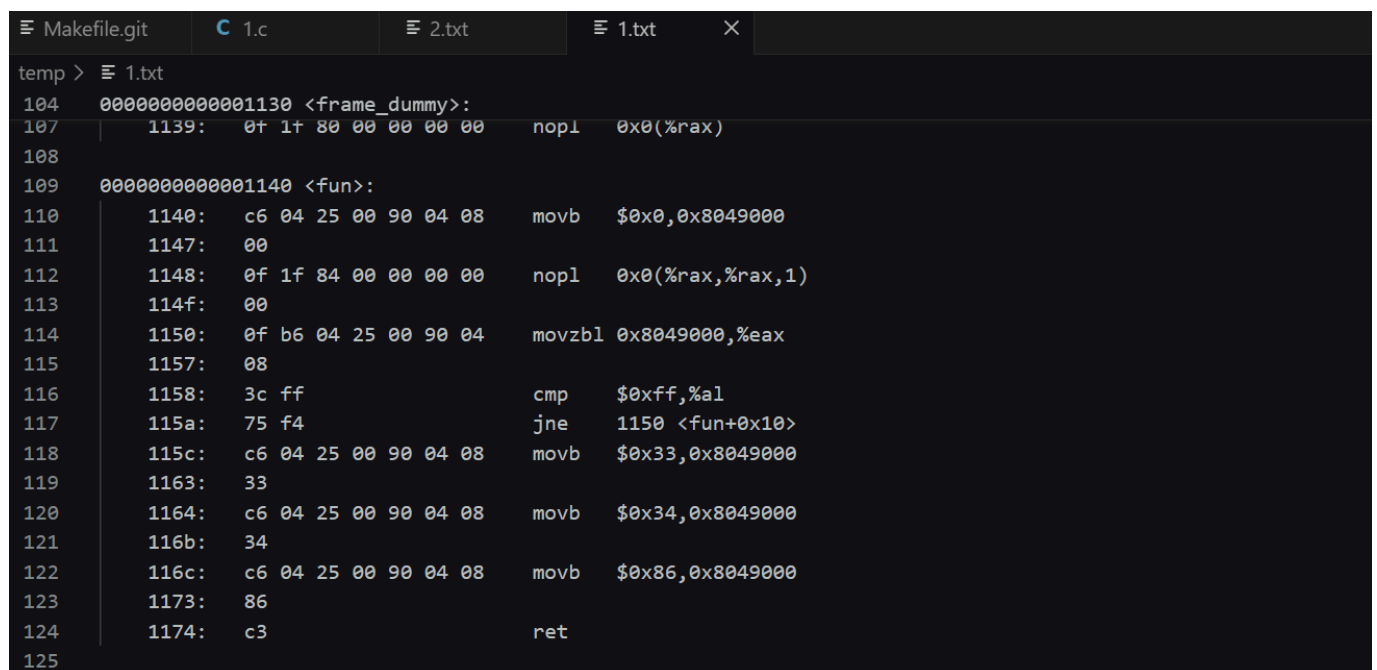
也许你从来都没听说过C语言中有 `volatile` 这个关键字, 但它从C语言诞生开始就一直存在. `volatile` 关键字的作用十分特别, 它的作用是避免编译器对相应代码进行优化. 你应该动手体会一下 `volatile` 的作用, 在GNU/Linux下编写以下代码:

```
void fun() {
    volatile unsigned char *p = (void *)0x8049000;
    *p = 0;
    while(*p != 0xff);
    *p = 0x33;
    *p = 0x34;
    *p = 0x86;
}
```

然后使用 `-O2` 编译代码. 尝试去掉代码中的 `volatile` 关键字, 重新使用 `-O2` 编译, 并对比去掉 `volatile` 前后反汇编结果的不同.

你或许会感到疑惑, 代码优化不是一件好事情吗? 为什么会有 `volatile` 这种奇葩的存在? 思考一下, 如果代码中的地址 `0x8049000` 最终被映射到一个设备寄存器, 去掉 `volatile` 可能会带来什么问题?

使用 `O2` 编译上面的程序, 并将结果反汇编, 得到如下:



```
temp > 1.txt
104 0000000000001130 <frame_dummy>:
107 1139: 0f 1f 80 00 00 00 00 nopl 0x0(%rax)
108
109 0000000000001140 <fun>:
110 1140: c6 04 25 00 90 04 08 movb $0x0,0x8049000
111 1147: 00
112 1148: 0f 1f 84 00 00 00 00 nopl 0x0(%rax,%rax,1)
113 114f: 00
114 1150: 0f b6 04 25 00 90 04 movzbl 0x8049000,%eax
115 1157: 08
116 1158: 3c ff cmp $0xff,%al
117 115a: 75 f4 jne 1150 <fun+0x10>
118 115c: c6 04 25 00 90 04 08 movb $0x33,0x8049000
119 1163: 33
120 1164: c6 04 25 00 90 04 08 movb $0x34,0x8049000
121 116b: 34
122 116c: c6 04 25 00 90 04 08 movb $0x86,0x8049000
123 1173: 86
124 1174: c3 ret
125
```

去掉 `volatile` 关键字后, 反汇编得到如下:


```
≡ Makefile.git  C 1.c  ≡ 2.txt  X  ≡ 1.txt
temp > ≡ 2.txt
104  0000000000001130 <frame_dummy>:
107  1139:  0+ 1+ 80 00 00 00 00  nopl  0x0(%rax)
108
109  0000000000001140 <fun>:
110  1140:  c6 04 25 00 90 04 08  movb  $0x0,0x8049000
111  1147:  00
112  1148:  eb fe  jmp  1148 <fun+0x8>
113
```

能看到，在使用 -O2 优化编译时，带有 volatile 的版本会在每次循环中都显式地从地址 0x8049000 重新读取数据，并根据读取结果判断是否退出循环，因此生成的汇编代码中可以看到反复的内存访问指令。

而去掉 volatile 后，编译器会假设该内存位置的值不会被外部改变，在这种语义下，*p 在赋值为 0 后，其值被认为始终不变，因此条件 *p != 0xff 恒为真，循环被优化为一个无限循环，后续对该地址的读取和写入操作也被全部消除，从而生成仅包含一次写操作和死循环的汇编代码。

而如果地址 0x8049000 实际上映射到一个设备寄存器时，硬件可能会异步修改该寄存器的值，但编译后的程序不再重新读取该地址，导致无法感知设备状态的变化，程序将永远停留在循环中，从而产生死循环。

4.实现IOE——时钟

4.1 实现 _uptime() 函数

在 nexus-am/am/arch/x86-nemu/src/ioe.c 里，inl(RTC_PORT) 表示从端口读取当前的时间计数值，boot_time 则是在系统启动时记录下来的初始时间，因此两者相减就得到了系统运行时长，如下：

```
14  unsigned long _uptime() {
15      return inl(RTC_PORT) - boot_time;
16  }
```

在 NEMU 中运行 timetest 程序，在 nexus-am/tests/timetest 目录下，执行

```
make ARCH=x86-nemu ALL=timetest run
```

运行结果如下：

```

SuYaolei@988d7f56373a:~/code/ics2017/nexus-am/tests/timetest$ make ARCH=x86-nemu ALL=timetest run
./build/nemu -l /home/SuYaolei/code/ics2017/nexus-am/tests/timetest/build/nemu-log.txt /home/SuYaolei/build/timetest-x86-nemu.bin
[src/monitor/diff-test/diff-test.c,97,init_diffptest] Connect to QEMU successfully
[src/monitor/monitor.c,65,load_img] The image is /home/SuYaolei/code/ics2017/nexus-am/tests/timetest/b
Welcome to NEMU!
[src/monitor/monitor.c,30,welcome] Build time: 19:52:26, Apr 23 2026
For help, type "help"
(nemu) c
1 second.
2 seconds.
3 seconds.
4 seconds.
5 seconds.
6 seconds.
7 seconds.
8 seconds.
9 seconds.
10 seconds.
11 seconds.
12 seconds.
13 seconds.

```

能看到成功输出了运行时间。

4.2 看看NEMU跑多快

先注释掉 nemu/include/common.h 中的 DEBUG 和 DIFF_TEST 宏，之后在 nexus-am/apps 目录下依次运行以下benchmark：

- dhrystone
- coremark
- microbench

运行结果依次如下：

```

SuYaolei@988d7f56373a:~/code/ics2017/nexus-am/apps/dhrystone$ make run
make: [/home/SuYaolei/code/ics2017/nexus-am/Makefile.compile:86: klib] Error 2 (ignored)
ld: warning: /home/SuYaolei/code/ics2017/nexus-am/apps/dhrystone/build/dhrystone-x86-nemu has a LOAD segment
make[1]: Entering directory '/home/SuYaolei/code/ics2017/nemu'
./build/nemu -l /home/SuYaolei/code/ics2017/nexus-am/apps/dhrystone/build/nemu-log.txt /home/SuYaolei/code/build/dhrystone-x86-nemu.bin
[src/monitor/monitor.c,65,load_img] The image is /home/SuYaolei/code/ics2017/nexus-am/apps/dhrystone/build/
Welcome to NEMU!
[src/monitor/monitor.c,30,welcome] Build time: 19:59:11, Apr 23 2026
For help, type "help"
(nemu) c
Dhrystone Benchmark, Version C, Version 2.2
Trying 500000 runs through Dhrystone.
Finished in 3960 ms
=====
Dhrystone PASS          260 Marks
                        vs. 100000 Marks (i7-6700 @ 3.40GHz)
nemu: HIT GOOD TRAP at eip = 0x00100071

```

```

SuYaoLei@988d7f56373a:~/code/ics2017/nexus-am/apps/coremark$ make run
ild/coremark-x86-nemu.bin
[src/monitor/monitor.c,65,load_img] The image is /home/SuYaoLei/code/ics2017/nexus-am/apps/coremark
Welcome to NEMU!
[src/monitor/monitor.c,30,welcome] Build time: 19:59:11, Apr 23 2026
For help, type "help"
(nemu) c
Running CoreMark for 1000 iterations
2K performance run parameters for coremark.
CoreMark Size      : 666
Total time (ms)    : 5255
Iterations         : 1000
Compiler version   : GCC14.2.0
seedcrc           : 0xe9f5
[0]crclist        : 0xe714
[0]crcmatrix      : 0x1fd7
[0]crcstate       : 0x8e3a
[0]crcfinal       : 0xd340
Finised in 5255 ms.
=====
CoreMark PASS      850 Marks
                   vs. 100000 Marks (i7-6700 @ 3.40GHz)
nemu: HIT GOOD TRAP at eip = 0x00100071

(nemu) █

```

```

SuYaoLei@988d7f56373a:~/code/ics2017/nexus-am/apps/microbench$ make run
[src/monitor/monitor.c,65,load_img] The image is /home/SuYaoLei/code/ics2017/nexus-am/apps/microbench/bui
Welcome to NEMU!
[src/monitor/monitor.c,30,welcome] Build time: 19:59:11, Apr 23 2026
For help, type "help"
(nemu) c
[qsrt] Quick sort: * Passed.
      min time: 348 ms [1585]
[queen] Queen placement: * Passed.
      min time: 466 ms [1107]
[bf] Brainf**k interpreter: * Passed.
      min time: 3092 ms [847]
[fib] Fibonacci number: * Passed.
      min time: 5955 ms [479]
[sieve] Eratosthenes sieve: * Passed.
      min time: 5356 ms [791]
[15pz] A* 15-puzzle search: * Passed.
      min time: 858 ms [675]
[dinic] Dinic's maxflow algorithm: * Passed.
      min time: 811 ms [1669]
[lzip] Lzip compression: * Passed.
      min time: 2749 ms [962]
[ssort] Suffix sort: * Passed.
      min time: 478 ms [1237]
[md5] MD5 digest: * Passed.
      min time: 4463 ms [439]
=====
MicroBench PASS    979 Marks
                   vs. 100000 Marks (i7-6700 @ 3.40GHz)
nemu: HIT GOOD TRAP at eip = 0x00100032

(nemu) █

```

5.实现IOE(2)——键盘

这里需要实现 `_read_key()` 从键盘控制器读取按键输入，先通过 `inb(I8042_STATUS_PORT)` 读取键盘控制器的状态寄存器，并用掩码 `I8042_STATUS_HASKEY_MASK` 判断当前是否有按键数据可读，如果没有数据，就返回

_KEY_NONE 表示没有按键事件，如果检测到有数据，则通过 inl(I8042_DATA_PORT) 从数据端口读取具体的按键值并返回。

```
5  #define I8042_DATA_PORT 0x60
6  #define I8042_STATUS_PORT 0x64
7  #define I8042_STATUS_HASKEY_MASK 0x1
```

```
53  int _read_key() {
54      if ((inb(I8042_STATUS_PORT) & I8042_STATUS_HASKEY_MASK) == 0) {
55          return _KEY_NONE;
56      }
57      return inl(I8042_DATA_PORT);
58  }
```

实现后，在 NEMU 中运行 keytest 程序(在nexus-am/tests/keytest目录下)，执行结果如下：

```
SuYaolei@988d7f56373a:~/code/ics2017/nexus-am/tests/keytest$ make run
ild/keytest-x86-nemu.bin
[src/monitor/monitor.c,65,load_img] The image is /home/SuYaolei/code/ics2017/nexus-am/tests/keytest/build/keyte
Welcome to NEMU!
[src/monitor/monitor.c,30,welcome] Build time: 19:59:11, Apr 23 2026
For help, type "help"
(nemu) c
Get key: 45 D down
Get key: 45 D up
Get key: 31 E down
Get key: 31 E up
Get key: 51 L down
Get key: 51 L up
Get key: 54 RETURN down
Get key: 54 RETURN up
Get key: 59 V down
Get key: 59 V up
Get key: 33 T down
Get key: 33 T up
```

可以看到输出了相应的按键信息。

问题：如何检测多个键同时被按下

在游戏中,很多时候需要判断玩家是否同时按下了多个键,例如RPG游戏中的八方向行走,格斗游戏中的组合招式等等.根据键盘码的特性,你知道这些功能是如何实现的吗?

键盘在每个按键按下时会发送一个按下码，松开时会发送对应的释放码。程序在接收到这些键盘码后，不是只处理当前这一按，而是维护一个数组，记录每个键当前是按下还是松开。当收到按下码时就把对应位置设为 1，收到释放码时设为 0。

这样一来，只需要同时检查多个键的状态位是否都为 1 就能判断多个键是否被同时按下。

6.VGA

6.1 添加内存映射I/O

在 nemu/src/memory/memory.c 中，需要在 paddr_read() 和 paddr_write() 中加入对内存映射I/O的判断. 通过 is_mmio() 函数判断一个物理地址是否被映射到 I/O空间, 如果是, is_mmio()会返回映射号, 否则返回-1，实现如

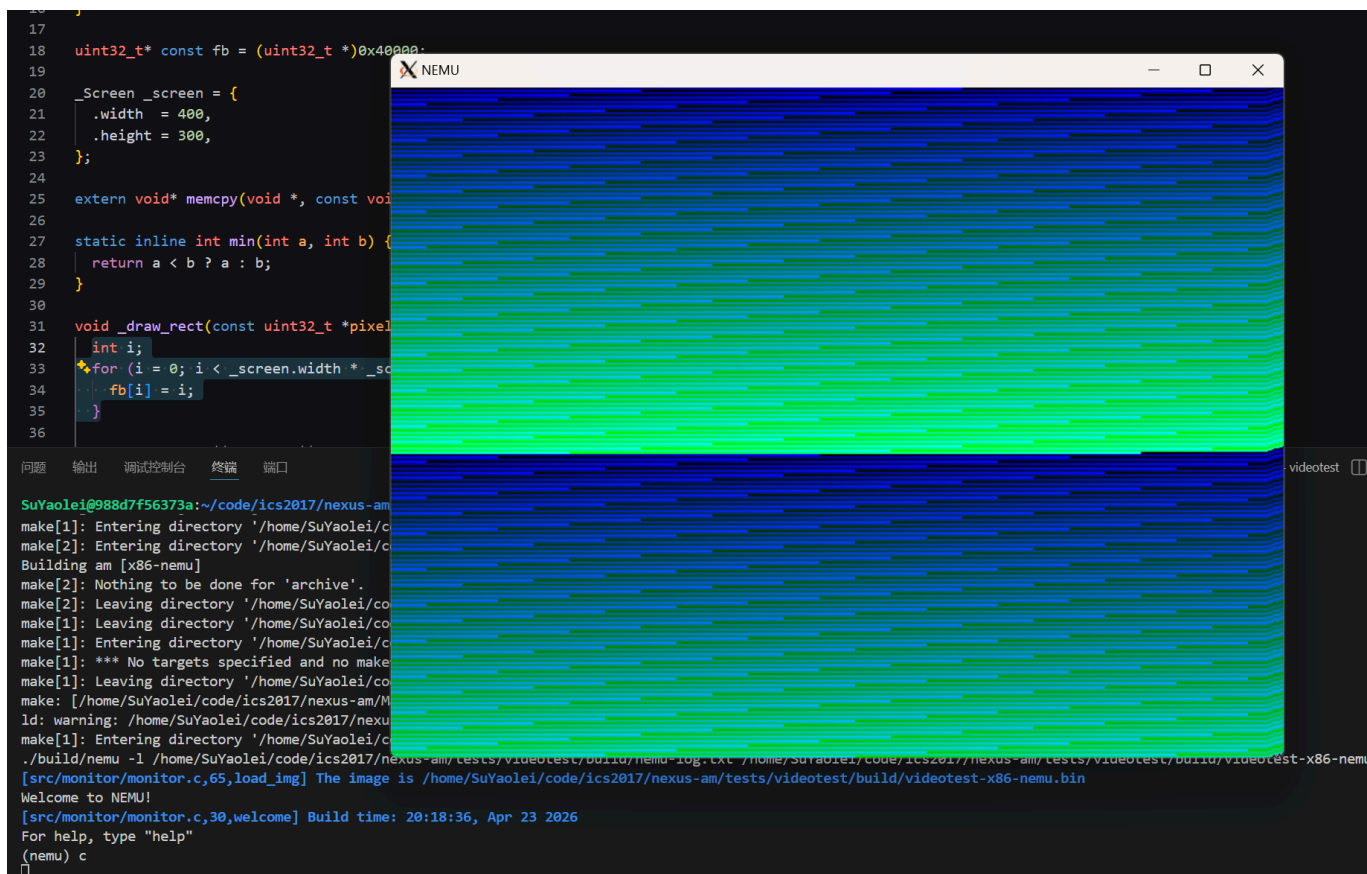
下:

```

15 uint32_t paddr_read(paddr_t addr, int len) {
16     int map_NO = is_mmio(addr);
17     if (map_NO != -1) {
18         return mmio_read(addr, len, map_NO);
19     }
20     return pmem_rw(addr, uint32_t) & (~0u >> ((4 - len) << 3));
21 }
22
23 void paddr_write(paddr_t addr, int len, uint32_t data) {
24     int map_NO = is_mmio(addr);
25     if (map_NO != -1) {
26         mmio_write(addr, len, data, map_NO);
27         return;
28     }
29     memcpy(guest_to_host(addr), &data, len);
30 }

```

实现后, 在 NEMU 中运行 videotest 程序(在nexus-am/tests/videotest目录下), 执行结果如下:



能看到成功输出了一些颜色信息。

6.2 实现IOE(3)

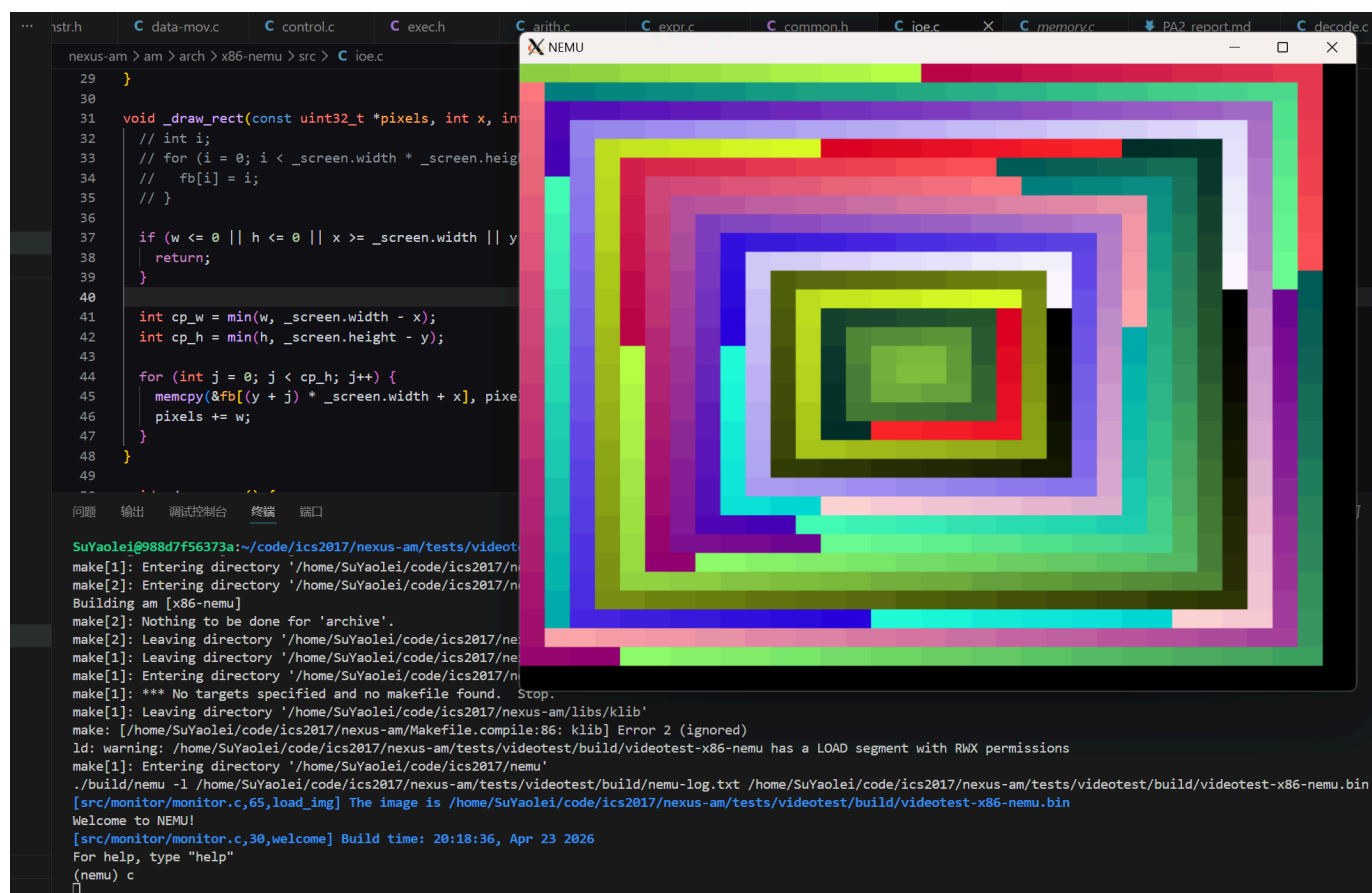
这里需要实现正确的 `_draw_rect()`，如下：

```

31 void _draw_rect(const uint32_t *pixels, int x, int y, int w, int h) {
32     // int i;
33     // for (i = 0; i < _screen.width * _screen.height; i++) {
34     //     fb[i] = i;
35     // }
36
37     if (w <= 0 || h <= 0 || x >= _screen.width || y >= _screen.height) {
38         return;
39     }
40
41     int cp_w = min(w, _screen.width - x);
42     int cp_h = min(h, _screen.height - y);
43
44     for (int j = 0; j < cp_h; j++) {
45         memcpy(&fb[(y + j) * _screen.width + x], pixels, cp_w * sizeof(uint32_t));
46         pixels += w;
47     }
48 }

```

再次运行 videotest 程序，显示结果如下：



6.3 运行打字小游戏

在 nexus-am/apps/typing 目录下执行

```
make run
```

结果如下：

```
str.h  C data-mov.c  C control.c  C exec.h  C arith.c  C expr.c  C common.h  C ioe.c  X  C memory.c  PA2_report.md  C d
nexus-am > am > arch > x86-nemu > src > C ioe.c
29 }
30
31 void _draw_rect(const uint32_t *pixels, int
32 // int i;
33 // for (i = 0; i < _screen.width * _scre
34 // fb[i] = i;
35 // }
36
37 if (w <= 0 || h <= 0 || x >= _screen.wid
38 return;
39 }
40
41 int cp_w = min(w, _screen.width - x);
42 int cp_h = min(h, _screen.height - y);
43
44 for (int j = 0; j < cp_h; j++) {
45 memcpy(&fb[(y + j) * _screen.width + x]
46 pixels += w;
47 }
48 }
49
--
问题 输出 调试控制台 终端 端口
SuYaoLei@988d7f56373a:~/code/ics2017/nexus-am/apps/
make[1]: Entering directory '/home/SuYaoLei/code/ic
make[2]: Entering directory '/home/SuYaoLei/code/ic
Building am [x86-nemu]
make[2]: Nothing to be done for 'archive'.
make[2]: Leaving directory '/home/SuYaoLei/code/ic
make[1]: Leaving directory '/home/SuYaoLei/code/ics
make[1]: Entering directory '/home/SuYaoLei/code/ics
make[1]: *** No targets specified and no makefile fi
make[1]: Leaving directory '/home/SuYaoLei/code/ics
make: [/home/SuYaoLei/code/ics2017/nexus-am/Makefil
ld: warning: /home/SuYaoLei/code/ics2017/nexus-am/a
make[1]: Entering directory '/home/SuYaoLei/code/ics2017/nemu'
./build/nemu -l /home/SuYaoLei/code/ics2017/nexus-am/apps/typing/build/nemu-log.txt /home/SuYaoLei/code/ics2017/nexus-am/apps/typing/build/typing-x86-nemu.bin
[src/monitor/monitor.c,65,load_img] The image is /home/SuYaoLei/code/ics2017/nexus-am/apps/typing/build/typing-x86-nemu.bin
Welcome to NEMU!
[src/monitor/monitor.c,30,welcome] Build time: 20:18:36, Apr 23 2026
For help, type "help"
(nemu) c
[]
```

能看到运行成功。

问题：在一些90年代的游戏里,很多渐出渐入效果都是通过调色板实现的,你知道其中的玄机吗?

在8位颜色模式下,每个像素存的是调色板索引,而不是RGB值。也就是说,如果我们逐步修改调色板中的颜色值,屏幕显示的画面就会整体发生变化,但像素数据本身不用改变,这样效率非常高。

比如做渐出效果时,可以不断把调色板中每个颜色的 R、G、B 分量按比例减小,比如每一帧都乘一个小于1的系数,这样所有颜色都会逐渐变黑,最终实现整个画面淡出,反过来,渐入就是从全黑调色板开始,逐步恢复到原来的调色板。

因此这样修改一次调色板,就等价于同时修改了屏幕上所有像素的颜色,不需要逐像素处理,从而在性能有限的机器上非常效。

四、必答题

1. 在nemu/include/cpu/rtl.h中,你会看到由staticinline开头定义的各种RTL指令函数.选择其中一个函数,分别尝试去掉static,去掉inline或去掉两者,然后重新进行编译,你会看到发生错误.请分别解释为什么会发生这些错误?你有办法证明你的想法吗?

选择 rtl_mv 作为例子,先去掉 static,运行程序 bit,得到结果如下:

```

SuYaoLei@988d7f56373a:~/code/ics2017/nexus-am/tests/cputest$ make ARCH=x86-nemu ALL=bit run
Building bit [x86-nemu]
Building am [x86-nemu]
make[2]: *** No targets specified and no makefile found. Stop.
[src/monitor/monitor.c,65,load_img] The image is /home/SuYaoLei/code/ics2017/nexus-am/tests/cputest/build/bit-x86-nemu.bin
Welcome to NEMU!
[src/monitor/monitor.c,30,welcome] Build time: 14:40:34, Apr 25 2026
For help, type "help"
(nemu) c
nemu: HIT GOOD TRAP at eip = 0x00100061

(nemu) q
bit

```

程序成功运行无报错。

之后修改 nemu 的 makefile 文件的编译选项，将 -Werror 删去，保留 rtl_mv 函数的 static，删去 inline，运行结果如下：

```

SuYaoLei@988d7f56373a:~/code/ics2017/nexus-am/tests/cputest$ make ARCH=x86-nemu ALL=bit run
./include/cpu/rtl.h:127:14: warning: 'rtl_mv' defined but not used [-Wunused-function]
 127 | static void rtl_mv(rtlreg_t* dest, const rtlreg_t *src1) {
      |
+ CC src/cpu/exec/exec.c
In file included from ./include/cpu/decode.h:6,
                  from ./include/cpu/exec.h:9,
                  from src/cpu/exec/exec.c:1:
./include/cpu/rtl.h:127:14: warning: 'rtl_mv' defined but not used [-Wunused-function]
 127 | static void rtl_mv(rtlreg_t* dest, const rtlreg_t *src1) {
      |
+ CC src/cpu/exec/special.c
In file included from ./include/cpu/decode.h:6,
                  from ./include/cpu/exec.h:9,
                  from src/cpu/exec/special.c:1:
./include/cpu/rtl.h:127:14: warning: 'rtl_mv' defined but not used [-Wunused-function]
 127 | static void rtl_mv(rtlreg_t* dest, const rtlreg_t *src1) {
      |
+ LD build/nemu
[src/monitor/monitor.c,65,load_img] The image is /home/SuYaoLei/code/ics2017/nexus-am/tests/cputest/build/bit-x86-nemu.bin
Welcome to NEMU!
[src/monitor/monitor.c,30,welcome] Build time: 14:40:34, Apr 25 2026
For help, type "help"
(nemu) c
nemu: HIT GOOD TRAP at eip = 0x00100061

(nemu) q

```

程序弹出警告，rtl_mv 函数定义后未被使用，但是还是成功运行。

最后，将 static 和 inline 都删去，运行结果如下：


```

SuYaolei@988d7f56373a:~/code/ics2017/nexus-am/tests/cputest$ make ARCH=x86-nemu ALL=bit run
me/SuYaolei/code/ics2017/nemu/./include/cpu/rtl.h:129: first defined here
/usr/bin/ld: build/obj/cpu/exec/system.o: in function `rtl_mv':
/home/SuYaolei/code/ics2017/nemu/./include/cpu/rtl.h:129: multiple definition of `rtl_mv'; build/obj/cpu/decode/m
me/SuYaolei/code/ics2017/nemu/./include/cpu/rtl.h:129: first defined here
/usr/bin/ld: build/obj/cpu/exec/cc.o: in function `rtl_mv':
/home/SuYaolei/code/ics2017/nemu/./include/cpu/rtl.h:129: multiple definition of `rtl_mv'; build/obj/cpu/decode/m
me/SuYaolei/code/ics2017/nemu/./include/cpu/rtl.h:129: first defined here
/usr/bin/ld: build/obj/cpu/exec/logic.o: in function `rtl_mv':
/home/SuYaolei/code/ics2017/nemu/./include/cpu/rtl.h:129: multiple definition of `rtl_mv'; build/obj/cpu/decode/m
me/SuYaolei/code/ics2017/nemu/./include/cpu/rtl.h:129: first defined here
/usr/bin/ld: build/obj/cpu/exec/data-mov.o: in function `rtl_mv':
/home/SuYaolei/code/ics2017/nemu/./include/cpu/rtl.h:129: multiple definition of `rtl_mv'; build/obj/cpu/decode/m
me/SuYaolei/code/ics2017/nemu/./include/cpu/rtl.h:129: first defined here
/usr/bin/ld: build/obj/cpu/exec/arith.o: in function `rtl_mv':
/home/SuYaolei/code/ics2017/nemu/./include/cpu/rtl.h:129: multiple definition of `rtl_mv'; build/obj/cpu/decode/m
me/SuYaolei/code/ics2017/nemu/./include/cpu/rtl.h:129: first defined here
/usr/bin/ld: build/obj/cpu/exec/control.o: in function `rtl_mv':
/home/SuYaolei/code/ics2017/nemu/./include/cpu/rtl.h:129: multiple definition of `rtl_mv'; build/obj/cpu/decode/m
me/SuYaolei/code/ics2017/nemu/./include/cpu/rtl.h:129: first defined here
/usr/bin/ld: build/obj/cpu/exec/exec.o: in function `rtl_mv':
/home/SuYaolei/code/ics2017/nemu/./include/cpu/rtl.h:129: multiple definition of `rtl_mv'; build/obj/cpu/decode/m
me/SuYaolei/code/ics2017/nemu/./include/cpu/rtl.h:129: first defined here
/usr/bin/ld: build/obj/cpu/exec/special.o: in function `rtl_mv':
/home/SuYaolei/code/ics2017/nemu/./include/cpu/rtl.h:129: multiple definition of `rtl_mv'; build/obj/cpu/decode/m
me/SuYaolei/code/ics2017/nemu/./include/cpu/rtl.h:129: first defined here
collect2: error: ld returned 1 exit status
make[2]: *** [Makefile:43: build/nemu] Error 1
make[1]: *** [/home/SuYaolei/code/ics2017/nexus-am/Makefile.app:35: run] Error 2
make: [Makefile:13: Makefile.bit] Error 2 (ignored)
bit

```

程序报错，无法正确运行。

来一个一个分析，首先去掉 static 但保留 inline 不会报错，此时函数因为有 inline 的存在，在 -O2 下，编译器通常会把这些小函数全部内联展开，因此不会真正生成一个对外可见的函数符号，也就不会在链接阶段发生冲突或缺失问题，所以程序可以正常编译运行。

其次，去掉 inline 但保留 static 也不会报错，但会出现“未使用”的 warning，这是因为函数因为有 static，作用域限制在文件内，但不再强制内联，编译器可能会为每个包含该头文件的 .c 文件各自生成一份函数实体，如果某些编译单元没有调用这个函数，就会触发 -Wunused-function 警告，但由于是 static，这些函数只在各自文件内部存在，不会引起链接冲突。

最后，同时去掉 static 和 inline 会导致程序报错，此时函数变成普通的全局函数定义，而这些定义又写在头文件里，被多个 .c 文件重复包含，相当于在多个编译单元中生成了多个同名的全局函数定义，根据 C 语言的链接规则，全局符号只能定义一次，因此在链接阶段会报 multiple definition 的错误，导致程序无法生成或执行。

2. 请描述你在nemu目录下敲入make后,make程序如何组织.c和.h文件,最终生成可执行文件nemu/build/nemu.(这个问题包括两个方面:Makefile的工作方式和编译链接的过程.)

工作方式:

当执行 make 时，make 会先读取 Makefile，找到默认目标，然后检查这个目标依赖的文件例如一系列 .o 文件是否存在或是否过期，如果某个依赖不存在，或者其时间戳比目标更新，make 就会按照规则中指定的命令去生成它。这个过程是递归进行的，如果 .o 文件本身也依赖 .c 和 .h 文件，make 会先去更新这些依赖，再生成 .o。

因此，Makefile 类似于构建一个依赖关系图，而 make 的工作就是沿着这张图，从底层源文件开始，逐步构建，直到最终生成目标文件。

编译链接过程：

打开 nemu 目录下的 Makefile 来进行分析，在 nemu 目录下执行 make run 时，首先根据 Makefile 中的设置，默认目标是 app，而 app 又依赖于最终的可执行文件 \$(BINARY)，即 build/nemu：

```
1  NAME = nemu
2  INC_DIR += ./include
3  BUILD_DIR ?= ./build
4  OBJ_DIR ?= $(BUILD_DIR)/obj
5  BINARY ?= $(BUILD_DIR)/$(NAME)

9  .DEFAULT_GOAL = app

33 app: $(BINARY)
```

接下来，检查生成 build/nemu 所需的依赖项。在 Makefile 中可以看到：

```
40 v $(BINARY): $(OBJS)
41     $(call git_commit, "compile")
42     @echo + LD $@
43     @$(LD) -O2 -o $@ $^ -lSDL2 -lreadline
44
```

build/nemu 依赖于一组目标文件 \$(OBJS)，这些目标文件是通过如下规则从源文件自动生成的：

```
17 # Files to be compiled
18 SRCS = $(shell find src/ -name "*.c")
19 OBJS = $(SRCS:src/%.c=$(OBJ_DIR)/%.o)
```

也就是说，Makefile 会扫描 src/ 目录下所有 .c 文件，并将它们一一映射为 build/obj/ 目录下对应的 .o 文件。

如果这些 .o 文件不存在，或者其依赖的 .c / .h 文件发生了变化，就会根据编译规则逐个生成：

```
21 # Compilation patterns
22 v $(OBJ_DIR)/%.o: src/%.c
23     @echo + CC $<
24     @mkdir -p $(dir $@)
25     @$(CC) $(CFLAGS) -c -o $@ $<
```

也就是调用编译器对每个 .c 文件进行编译，生成对应的目标文件 .o。

当所有 .o 文件都准备好之后，执行链接步骤，将这些目标文件合并为最终的可执行文件：

```
40 v $(BINARY): $(OBJS)
41     $(call git_commit, "compile")
42     @echo + LD $@
43     @$(LD) -O2 -o $@ $^ -lSDL2 -lreadline
44
```

五、实验过程中的问题与解决方法

PA2 的工作量相比 PA1 还是比较大，关键是翻译那些指令的译码规则。

对于遇到的问题，首先是译码辅助函数选择不当，尤其是 I 与 SI 混用，它们的区别是 I 是无符号解释，SI 是有符号解释，比如一些指令要求把8位立即数符号扩展到操作数宽度参与计算，如果错用 decode_I，在涉及负立

即数、循环边界和地址位移的程序里会出现偏差。这个时候就可以把这类编码统一走 SI2E 或 SI_E2G，并在 decode_SI 中明确做符号扩展。

其次是EFLAGS相关的，对于 CF/OF，很多简单程序主要依赖 ZF，例如 adc/sbb 若直接沿用 add/sub 的判定思路，很容易在长整型用例中出错，原因在于这两条指令引入了旧CF参与运算，进位/借位判定必须在更高精度中间结果上完成，因此需要按操作数位宽做mask，再用64位中间变量抽取进位位，类似于模拟硬件加法器的进位链。

还有一些rtl函数实现的问题，比如 rtl_sext() 需要根据操作数的宽度（1、2、4 字节）对应来进行正确的符号扩展，如果没有的话，后续依赖符号扩展的指令，如 movsx、idiv 或 cltd，就会得到错误的操作数，从而导致整个运算结果错误；而 rtl_eq0 或 rtl_neq0 如果返回的值不是严格的 0 或 1，会直接影响到条件判断相关的指令，如 setcc 或各类条件跳转，使程序控制流出现偏差。（最后，在你写完 make_EHelper(...) 后，一定要在 all-instr.h 里添加声明...）